SESSION 2 · WHERE WE STOPPED

Last time: a loss for a latent-variable model

$$\mathcal{L}(x) = \underbrace{-\log p_\theta(x \mid z)}_{\text{reconstruction}} + \underbrace{\mathrm{KL}\big(q_\varphi(z \mid x) \,\|\, \mathcal{N}(0, I)\big)}_{\text{cloud vs. prior}}, \qquad z \sim q_\varphi(z \mid x)$$

122.3
reconstruction for our 3

+ 6.97
KL for our 3

= 129.2
its loss, one draw of z

Why it is principled: $-\mathcal{L}$ is a lower bound on $\log p_\theta(x)$, short by $\mathrm{KL}(q \,\|\, \text{true posterior})$.

Still missing: how to train it. The z in the middle is a random draw.

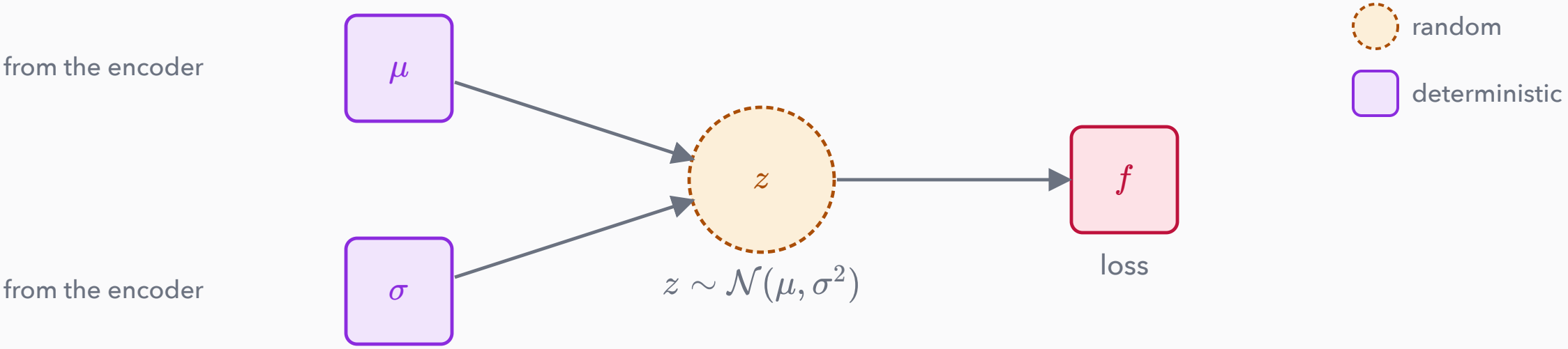
PART 5 · TOOL 3

Backpropagating through a random draw

The loss uses a z sampled from the encoder's cloud. To train the encoder, gradients must get from the loss back to μ and σ.

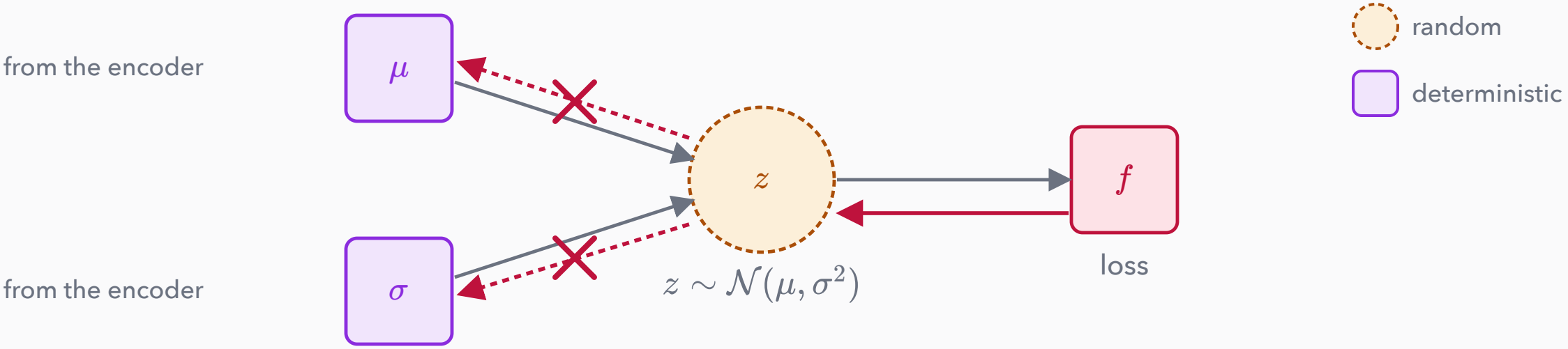
THE PROBLEM

A sampling step sits in the middle of the graph



THE PROBLEM

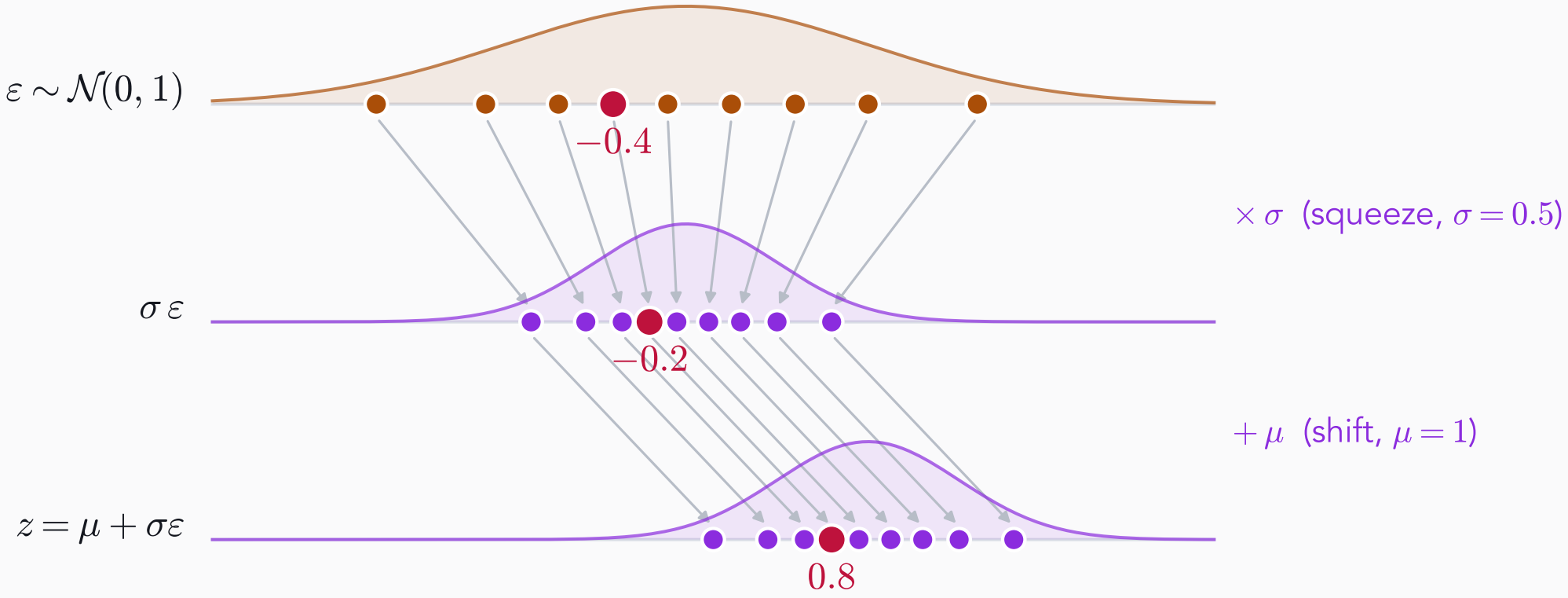
A sampling step sits in the middle of the graph



the loss knows how z should move, but z is a fresh random draw: how does it change when μ or σ changes a little?

THE TRICK

Draw standard noise first, then stretch and shift it

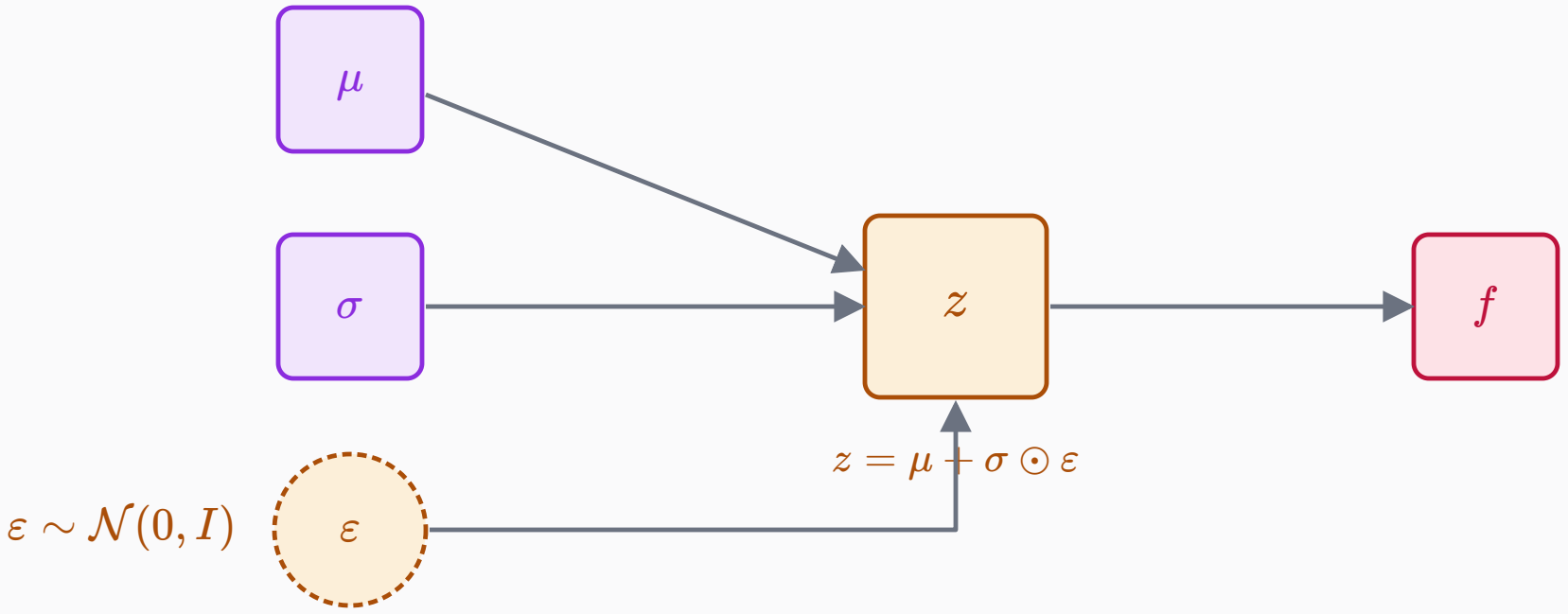


$$z = \mu + \sigma\,\varepsilon, \quad \varepsilon \sim \mathcal{N}(0,1) \quad \Longrightarrow \quad z \sim \mathcal{N}(\mu, \sigma^2)$$

> **Same distribution as before. But the randomness now enters as an input, ε, that does not depend on any weights.**

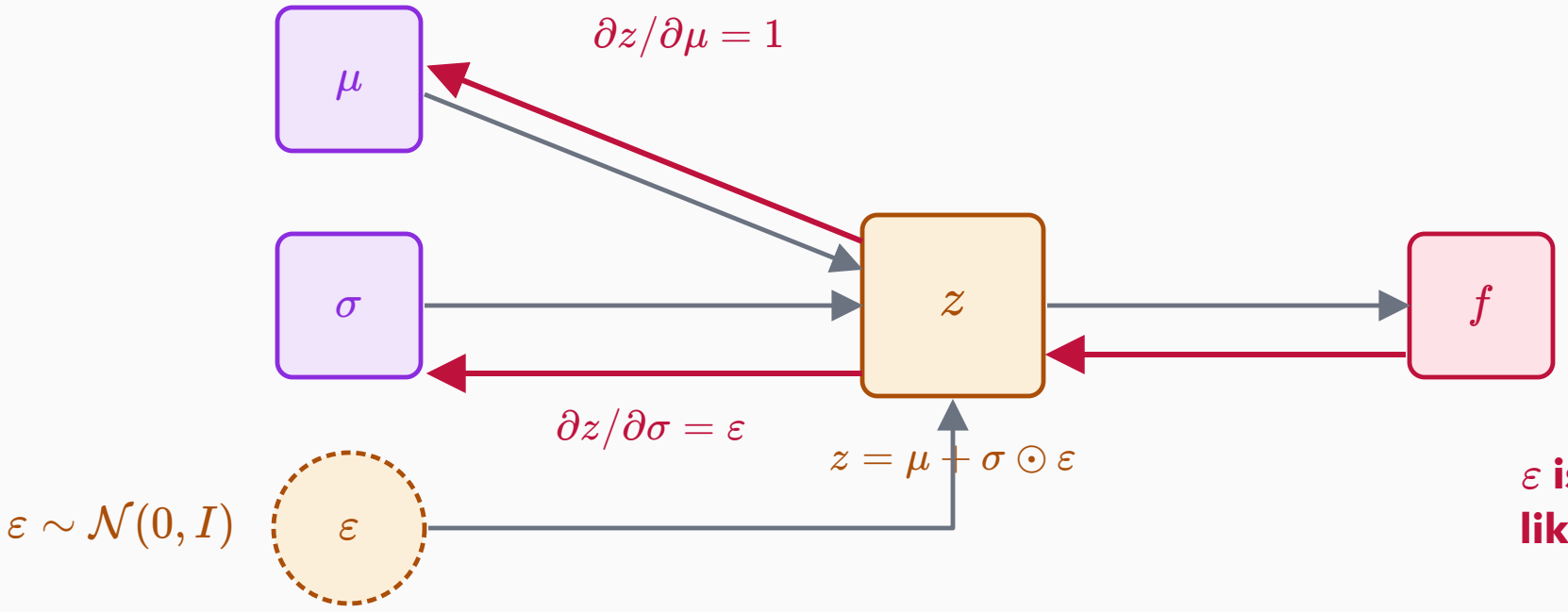
THE FIX

Now the path from the loss to μ and σ is ordinary arithmetic



THE FIX

Now the path from the loss to μ and σ is ordinary arithmetic



ε is just an input, like a pixel value

Take $f(z) = z^2$ as the loss

$$z = \mu + \sigma\varepsilon \qquad \Rightarrow \qquad \frac{\partial z}{\partial \mu} = 1, \qquad \frac{\partial z}{\partial \sigma} = \varepsilon, \qquad \frac{\partial f}{\partial z} = 2z$$

Take $f(z) = z^2$ as the loss

$$z = \mu + \sigma\varepsilon \qquad \Rightarrow \qquad \frac{\partial z}{\partial \mu} = 1, \qquad \frac{\partial z}{\partial \sigma} = \varepsilon, \qquad \frac{\partial f}{\partial z} = 2z$$

$$\frac{\partial f}{\partial \mu} = \frac{\partial f}{\partial z} \cdot \frac{\partial z}{\partial \mu} = 2z \cdot 1$$

Take $f(z) = z^2$ as the loss

$$z = \mu + \sigma\varepsilon \qquad \Rightarrow \qquad \frac{\partial z}{\partial \mu} = 1, \qquad \frac{\partial z}{\partial \sigma} = \varepsilon, \qquad \frac{\partial f}{\partial z} = 2z$$

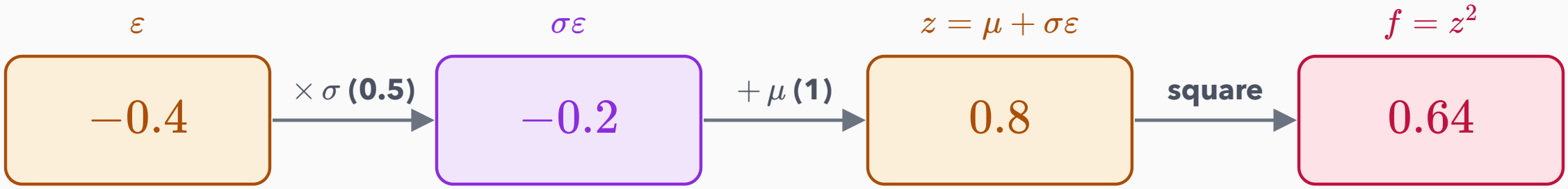
$$\frac{\partial f}{\partial \mu} = \frac{\partial f}{\partial z} \cdot \frac{\partial z}{\partial \mu} = 2z \cdot 1$$

$$\frac{\partial f}{\partial \sigma} = \frac{\partial f}{\partial z} \cdot \frac{\partial z}{\partial \sigma} = 2z \cdot \varepsilon$$

The noise value ε appears in the σ-gradient: a draw far from the centre pushes σ harder.

NOW PLUG IN NUMBERS

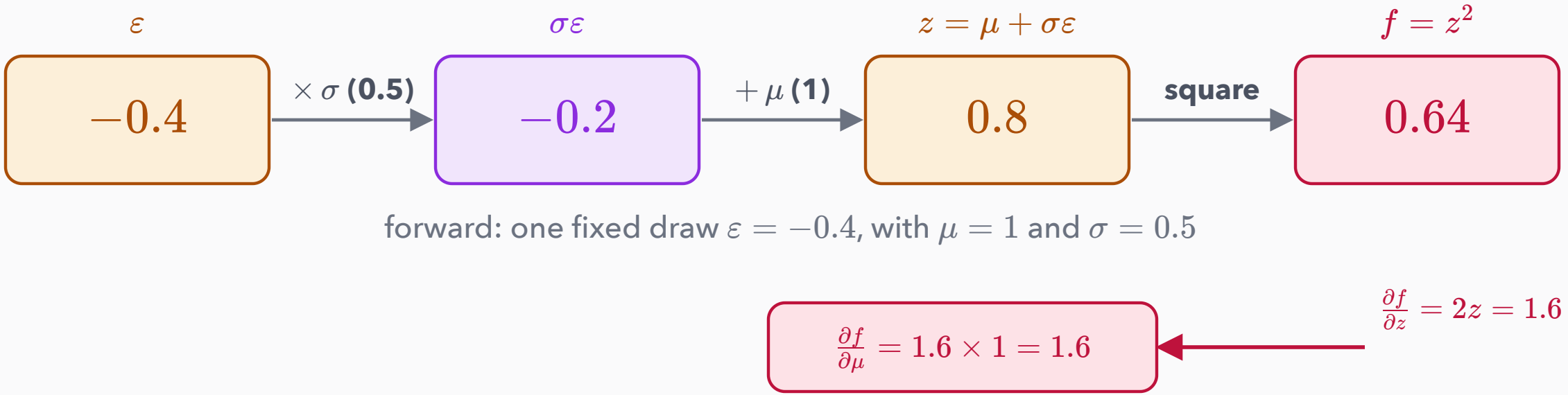
$$\mu = 1,\ \sigma = 0.5,\ \varepsilon = -0.4$$



forward: one fixed draw $\varepsilon = -0.4$, with $\mu = 1$ and $\sigma = 0.5$

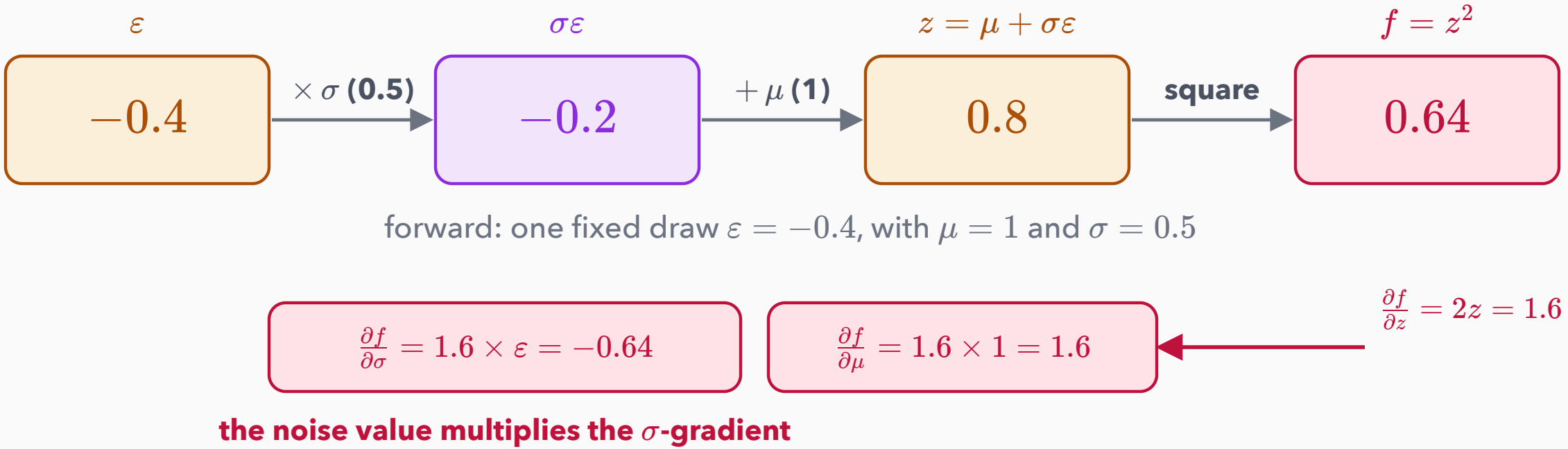
NOW PLUG IN NUMBERS

$$\mu = 1,\ \sigma = 0.5,\ \varepsilon = -0.4$$



NOW PLUG IN NUMBERS

$$\mu = 1,\ \sigma = 0.5,\ \varepsilon = -0.4$$



the noise value multiplies the σ-gradient

ONE MORE LINK IN THE CHAIN

The network outputs $v = \log \sigma^2$, not σ

$$\sigma = e^{v/2}$$

v can be any real number; σ is always positive

ONE MORE LINK IN THE CHAIN

The network outputs $v = \log \sigma^2$, not σ

$$\sigma = e^{v/2}$$

v can be any real number; σ is always positive

$$\frac{\partial \sigma}{\partial v} = \frac{1}{2} e^{v/2} = \frac{\sigma}{2} = 0.25$$

derivative of $e^{v/2}$

ONE MORE LINK IN THE CHAIN

The network outputs $v = \log \sigma^2$, not σ

$$\sigma = e^{v/2}$$

v can be any real number; σ is always positive

$$\frac{\partial \sigma}{\partial v} = \tfrac{1}{2} e^{v/2} = \frac{\sigma}{2} = 0.25$$

derivative of $e^{v/2}$

$$\frac{\partial f}{\partial v} = \frac{\partial f}{\partial \sigma} \cdot \frac{\partial \sigma}{\partial v} = -0.64 \times 0.25 = -0.16$$

one more chain-rule factor

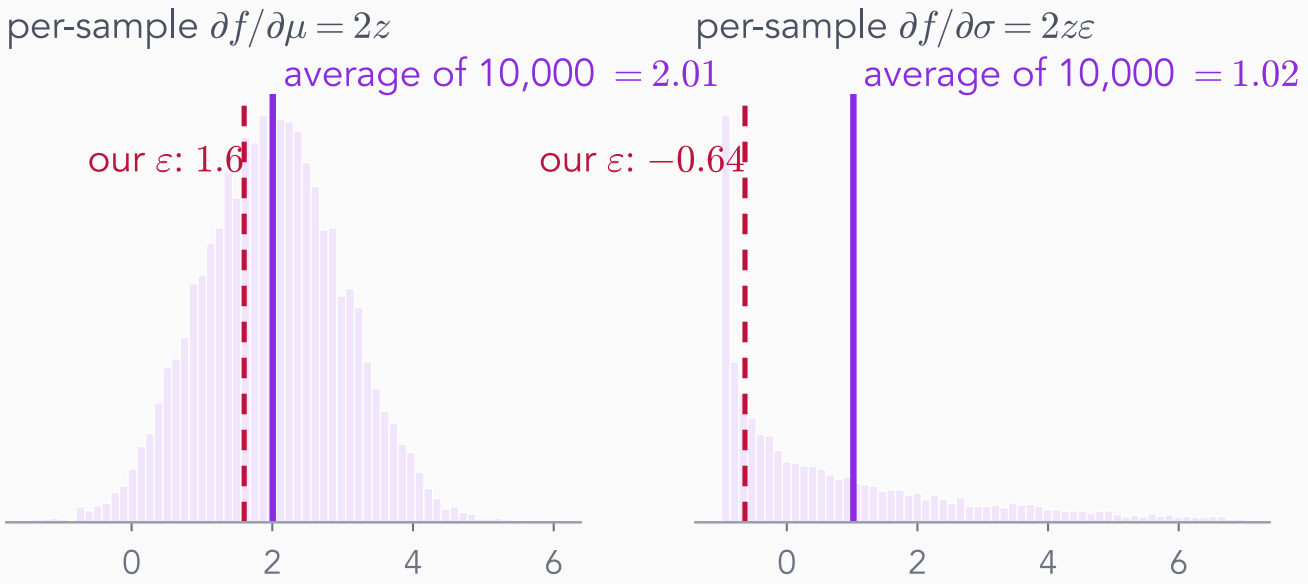
Autograd does exactly this. The point of doing it by hand once: no step is magic.

One draw is noisy; the average is exactly right

$$\mathbb{E}_\varepsilon\big[f\big] = \mathbb{E}\big[(\mu + \sigma\varepsilon)^2\big] = \mu^2 + \sigma^2 \quad \Rightarrow \quad \frac{\partial}{\partial\mu}\mathbb{E}[f] = 2\mu = 2, \qquad \frac{\partial}{\partial\sigma}\mathbb{E}[f] = 2\sigma = 1$$

One draw is noisy; the average is exactly right

$$\mathbb{E}_\varepsilon[f] = \mathbb{E}\left[(\mu+\sigma\varepsilon)^2\right] = \mu^2+\sigma^2 \quad \Rightarrow \quad \frac{\partial}{\partial\mu}\mathbb{E}[f] = 2\mu = 2, \qquad \frac{\partial}{\partial\sigma}\mathbb{E}[f] = 2\sigma = 1$$



Our single ε gave 1.6 and -0.64. Averaged over 10000 draws: 2.01 and 1.02. SGD only needs the right answer on average.

IN PYTORCH

Three lines

```
# encoder outputs, shape [B, 2] each
mu, logvar = encoder(x)
std = torch.exp(0.5 * logvar)     # sigma = exp(v / 2)
eps = torch.randn_like(std)       # standard normal, no grad
z   = mu + std * eps              # differentiable in mu, std
```

IN PYTORCH

Three lines

```
# encoder outputs, shape [B, 2] each
mu, logvar = encoder(x)
std = torch.exp(0.5 * logvar)      # sigma = exp(v / 2)
eps = torch.randn_like(std)        # standard normal, no grad
z   = mu + std * eps               # differentiable in mu, std
```

Why output $\log \sigma^2$? It can be any real number; σ must be positive.

IN PYTORCH

Three lines

```
# encoder outputs, shape [B, 2] each
mu, logvar = encoder(x)
std = torch.exp(0.5 * logvar)      # sigma = exp(v / 2)
eps = torch.randn_like(std)        # standard normal, no grad
z   = mu + std * eps               # differentiable in mu, std
```

Why output $\log \sigma^2$? It can be any real number; σ must be positive.

IN PYTORCH

Three lines

```
# encoder outputs, shape [B, 2] each
mu, logvar = encoder(x)
std = torch.exp(0.5 * logvar)      # sigma = exp(v / 2)
eps = torch.randn_like(std)        # standard normal, no grad
z   = mu + std * eps               # differentiable in mu, std
```

Why output $\log \sigma^2$? It can be any real number; σ must be positive.

`Normal(mu, std).rsample()` does the same thing; `.sample()` drops the gradient.

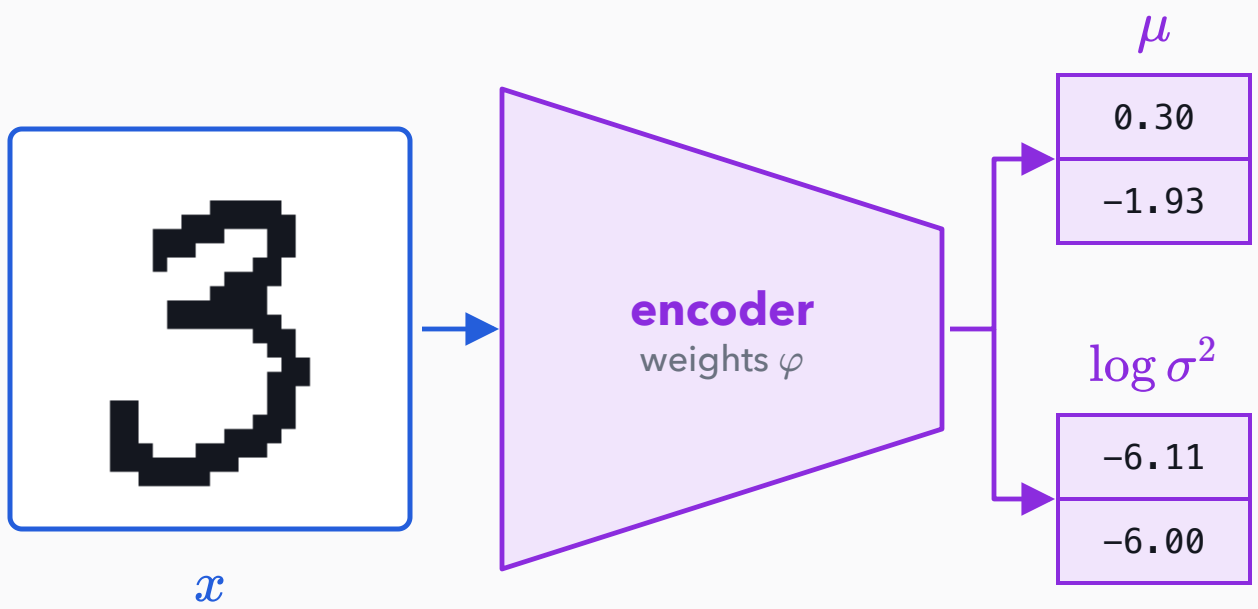
For our 3: $\mu = (0.3,\ -1.93)$, $\varepsilon = (0.62,\ -1.15)$ gives $z = (0.33,\ -1.99)$.

PART 6

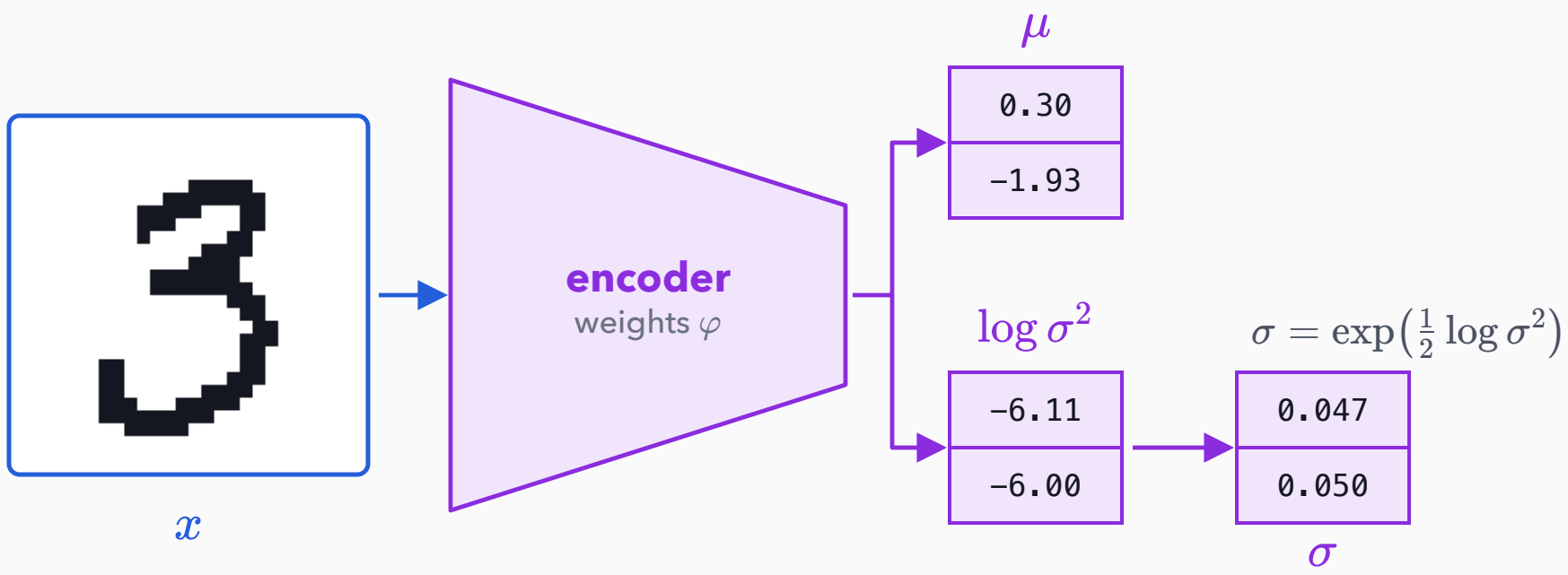
The variational autoencoder, end to end

Every piece is now in place. Follow our 3 through the trained network, then see what the model can generate.

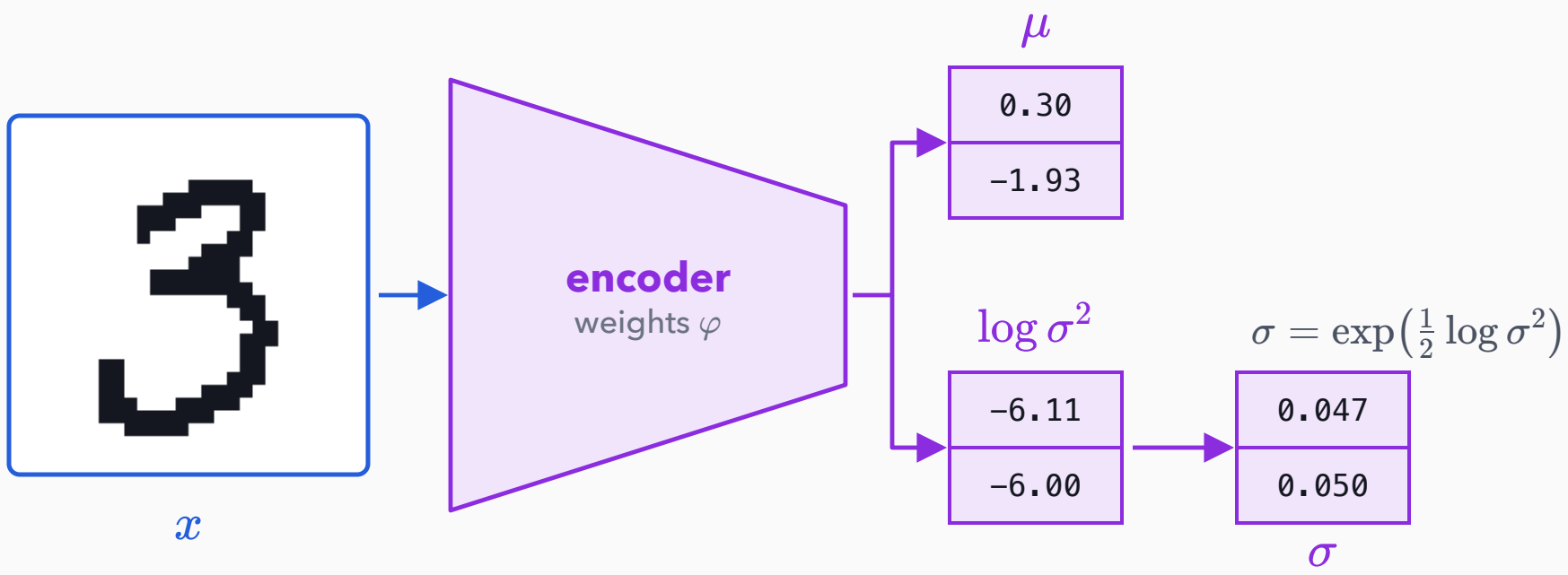
Encode: the image becomes a cloud



Encode: the image becomes a cloud



Encode: the image becomes a cloud

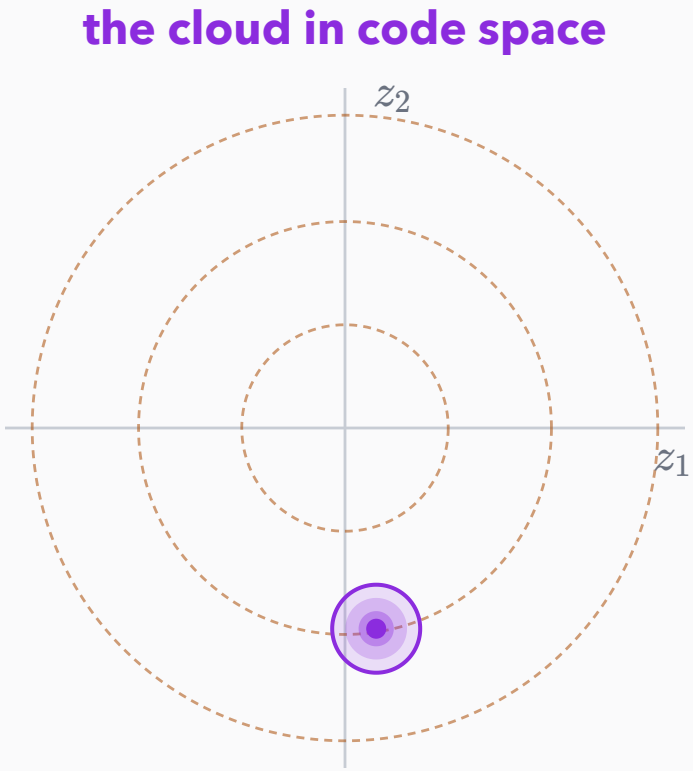


a cloud centred at μ with width σ:

$$q_\varphi(z \mid x) = \mathcal{N}(\mu, \sigma^2)$$

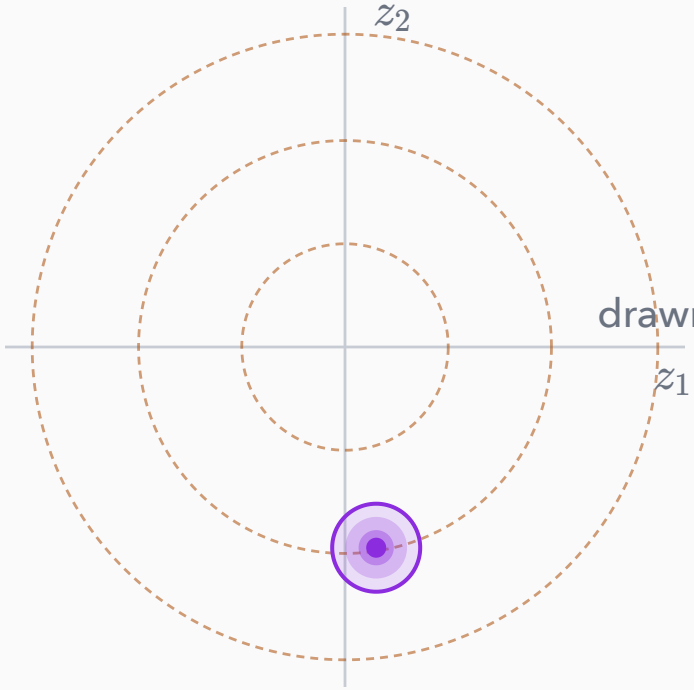
$\sigma \approx 0.05$ against the prior's $\sigma = 1$: the encoder is very sure where our 3 belongs.

Sample: one code from inside the cloud



Sample: one code from inside the cloud

the cloud in code space



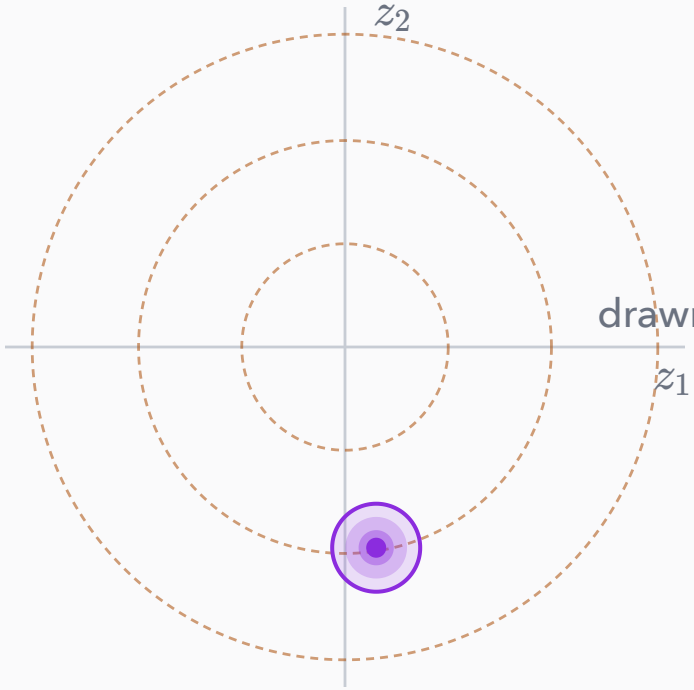


$\varepsilon = (0.62,\ -1.15)$

drawn from a standard normal once

Sample: one code from inside the cloud

the cloud in code space



ε

$\varepsilon = (0.62,\ -1.15)$

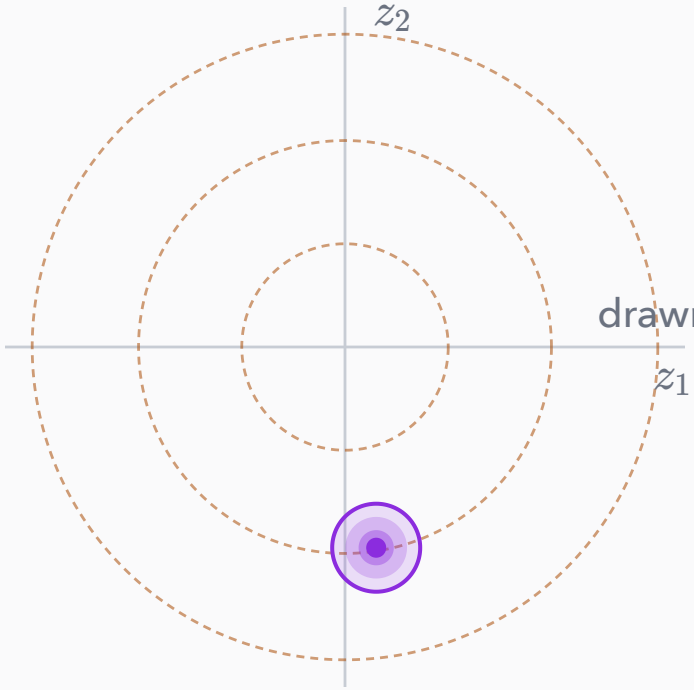
drawn from a standard normal once

$$z_1 = \mu_1 + \sigma_1 \varepsilon_1$$

$$= 0.30 + 0.047 \times 0.62 = 0.33$$

Sample: one code from inside the cloud

the cloud in code space



ε

$\varepsilon = (0.62,\ -1.15)$

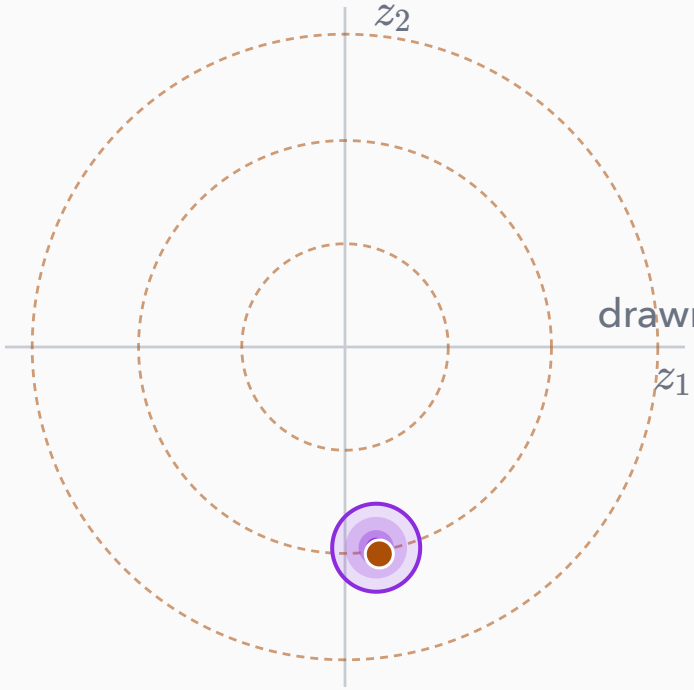
drawn from a standard normal once

$$z_1 = \mu_1 + \sigma_1 \varepsilon_1$$
$$= 0.30 + 0.047 \times 0.62 = 0.33$$

$$z_2 = \mu_2 + \sigma_2 \varepsilon_2$$
$$= -1.93 + 0.050 \times (-1.15) = -1.99$$

Sample: one code from inside the cloud

the cloud in code space



ε

$\varepsilon = (0.62,\ -1.15)$

drawn from a standard normal once

$$z_1 = \mu_1 + \sigma_1 \varepsilon_1$$
$$= 0.30 + 0.047 \times 0.62 = 0.33$$

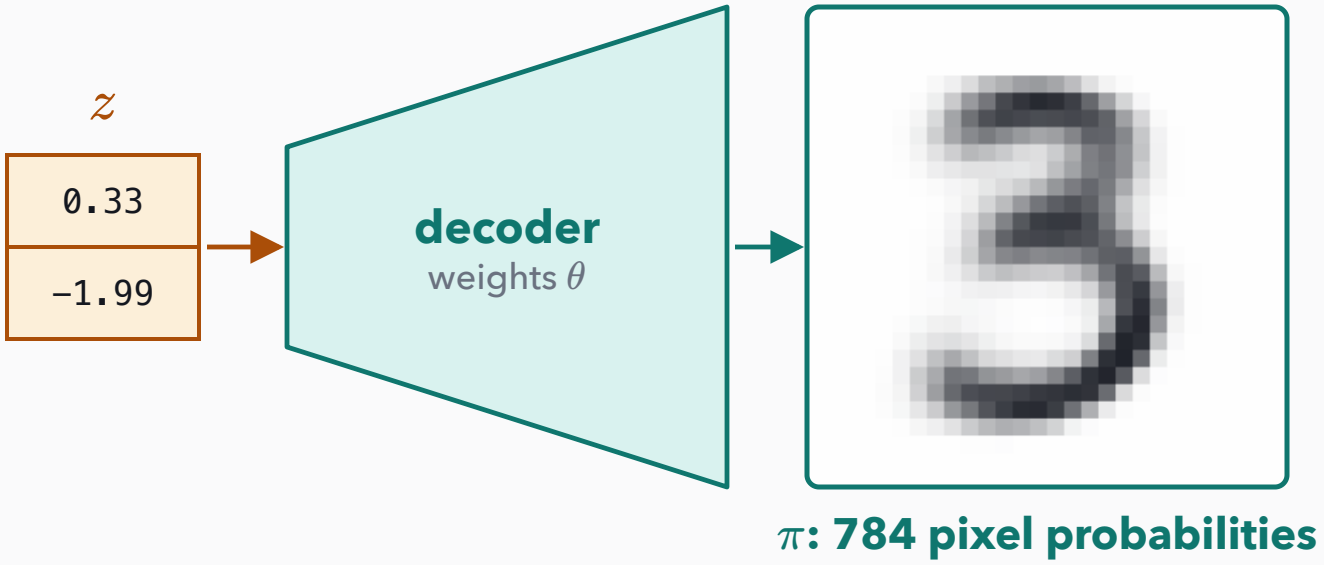
$$z_2 = \mu_2 + \sigma_2 \varepsilon_2$$
$$= -1.93 + 0.050 \times (-1.15) = -1.99$$

$z =$

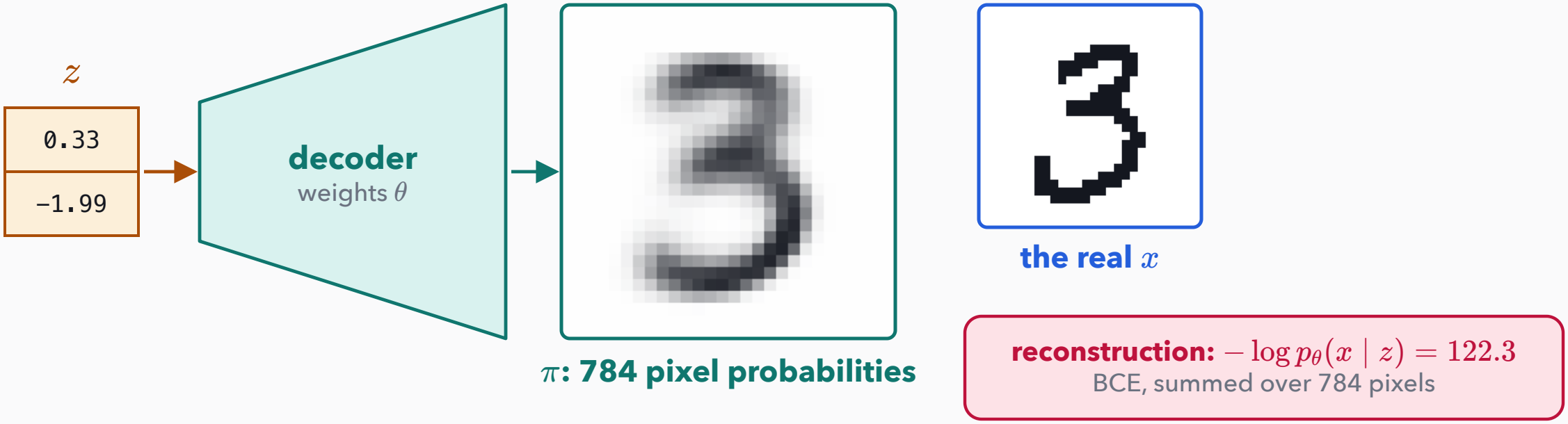
0.33	-1.99

one code from inside the cloud

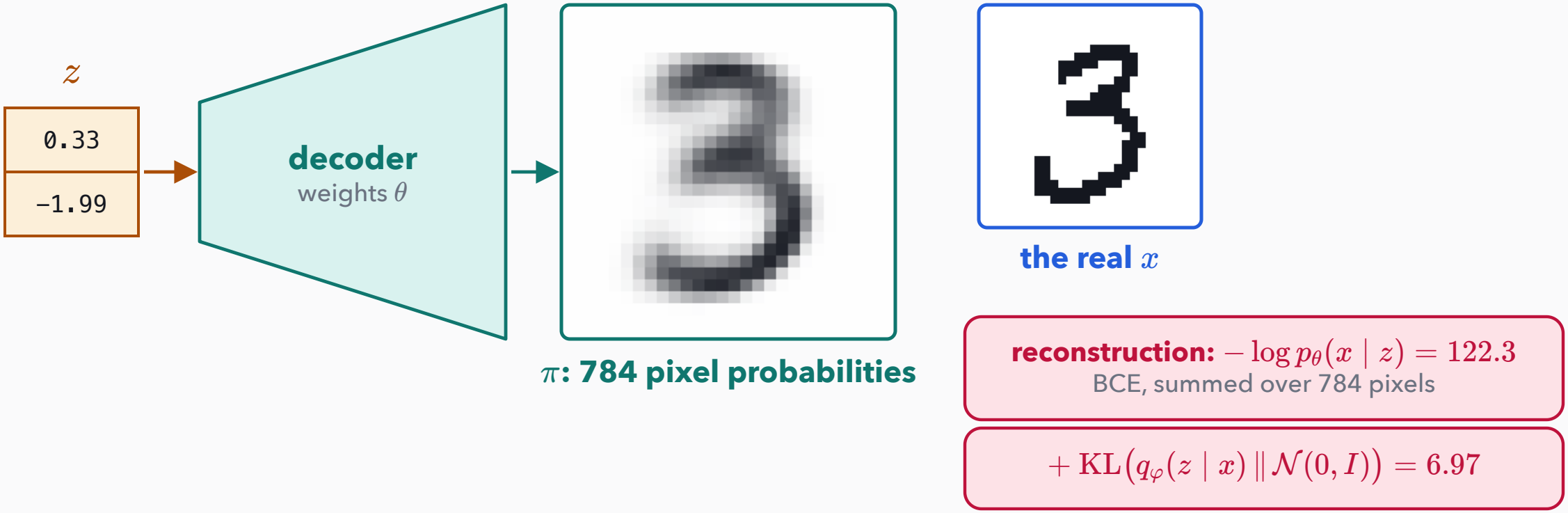
Decode, then score against the real 3



Decode, then score against the real 3



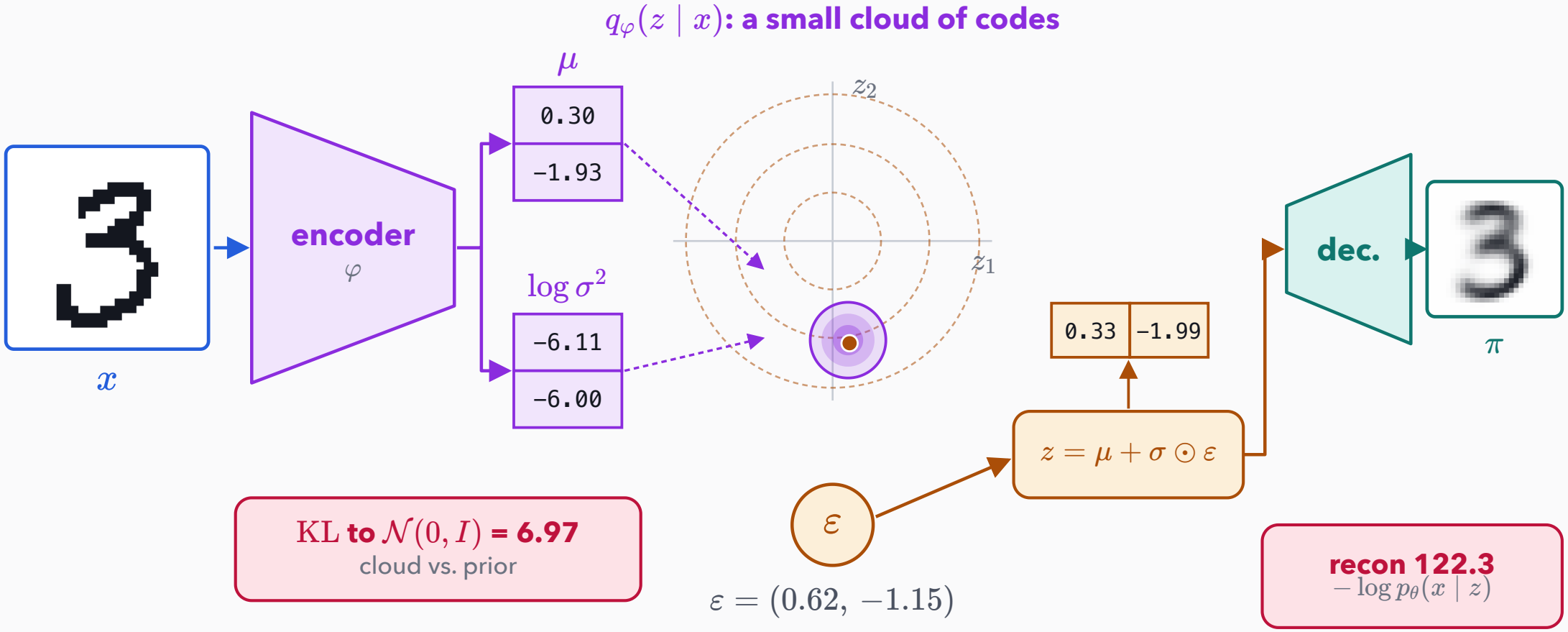
Decode, then score against the real 3



Loss for our 3: $122.3 + 6.97 = 129.2$ nats. Backpropagate it into both θ and φ.

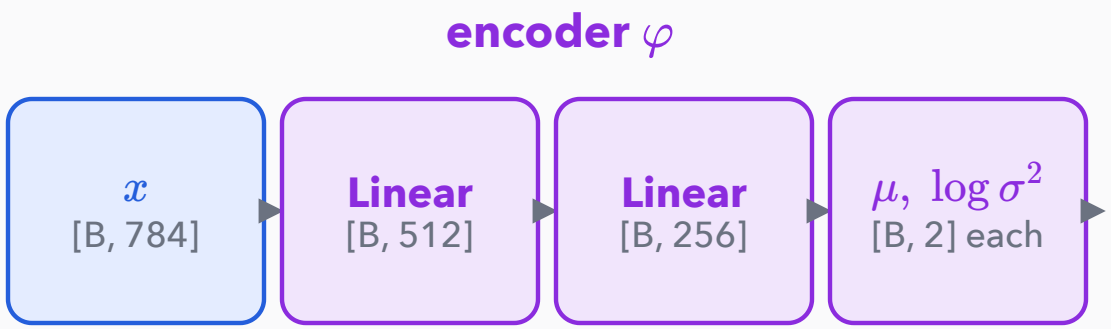
ALL THREE STEPS ON ONE PAGE

The whole VAE, for our 3



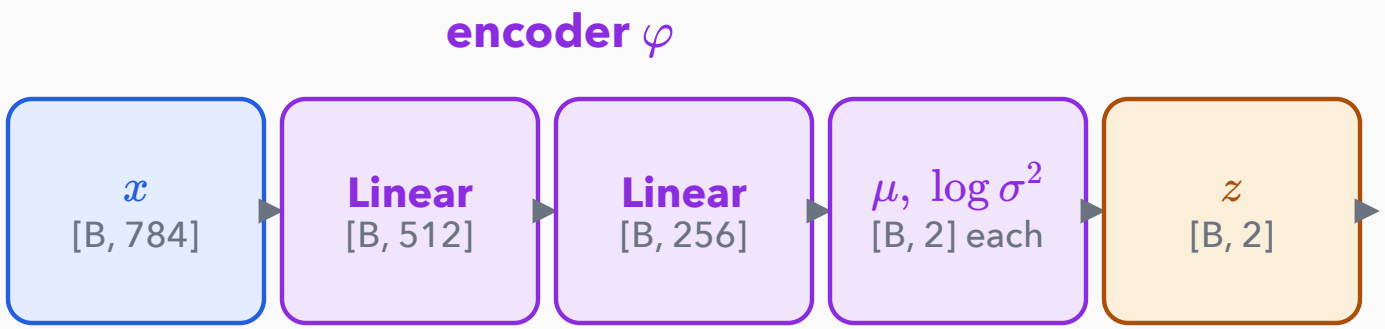
THE SAME NETWORK AS TENSORS

Shapes, layer by layer



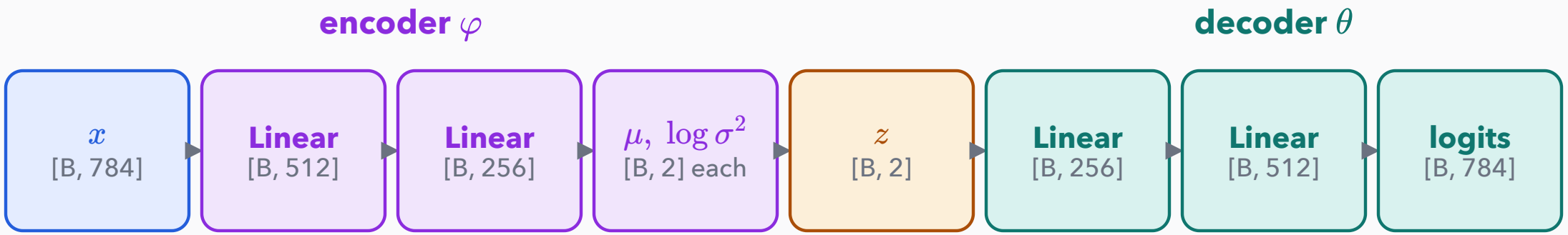
THE SAME NETWORK AS TENSORS

Shapes, layer by layer



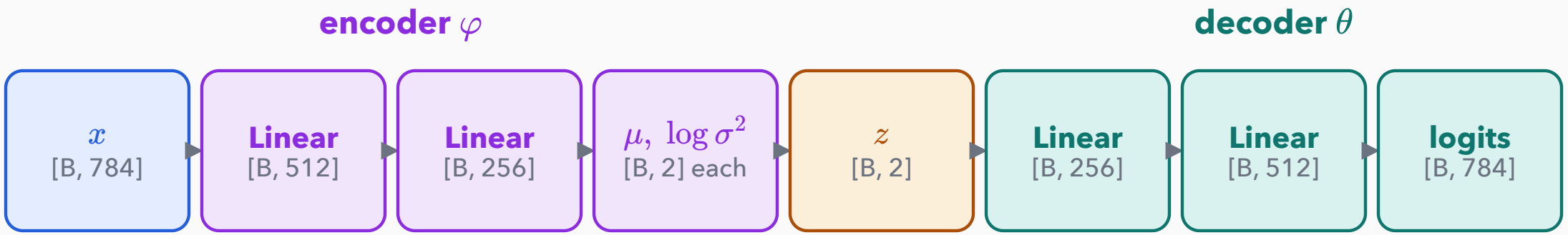
THE SAME NETWORK AS TENSORS

Shapes, layer by layer



THE SAME NETWORK AS TENSORS

Shapes, layer by layer



B = batch size. Only the 2 in the middle is the code; ε has the same shape as μ.

Loss per image: BCE summed over 784 pixels + KL summed over 2 coordinates.
Then average over the B images in the batch.

THE KL TERM FOR OUR 3

A tight cloud is expensive

$$\mathrm{KL} = \tfrac{1}{2} \sum_{j} \left(\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2 \right)$$

coordinate	μ_j	σ_j	μ_j^2	$\sigma_j^2 - 1$	$-\log \sigma_j^2$	$\frac{1}{2}(\text{sum})$

THE KL TERM FOR OUR 3

A tight cloud is expensive

$$\mathrm{KL} = \tfrac{1}{2} \sum_{j} \left(\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2 \right)$$

coordinate	μ_j	σ_j	μ_j^2	$\sigma_j^2 - 1$	$-\log \sigma_j^2$	$\frac{1}{2}(\text{sum})$
z_1	0.3	0.047	0.09	-1.00	6.11	**2.6**

THE KL TERM FOR OUR 3

A tight cloud is expensive

$$\mathrm{KL} = \tfrac{1}{2} \sum_{j} \left(\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2 \right)$$

coordinate	μ_j	σ_j	μ_j^2	$\sigma_j^2 - 1$	$-\log \sigma_j^2$	$\frac{1}{2}(\text{sum})$
z_1	0.3	0.047	0.09	-1.00	6.11	**2.6**
z_2	−1.93	0.05	3.72	-1.00	6.00	**4.37**

THE KL TERM FOR OUR 3

A tight cloud is expensive

$$\mathrm{KL} = \tfrac{1}{2}\sum_{j}\left(\mu_j^2 + \sigma_j^2 - 1 - \log\sigma_j^2\right)$$

coordinate	μ_j	σ_j	μ_j^2	$\sigma_j^2 - 1$	$-\log\sigma_j^2$	$\frac{1}{2}(\text{sum})$
z_1	0.3	0.047	0.09	-1.00	6.11	**2.6**
z_2	−1.93	0.05	3.72	-1.00	6.00	**4.37**
total						**6.97 nats**

$$-\log\sigma_1^2 = -\log(0.047^2) = -\log 0.0022 = 6.11$$

> **Most of the cost is $-\log\sigma^2$: the price of being precise. Reconstruction wants precision; the KL resists.**

ONE TRAINING STEP

The whole algorithm fits on a slide

```
mu, logvar = model.encoder(x)              # [B,2], [B,2]
z = mu + torch.exp(0.5*logvar) * torch.randn_like(mu)
logits = model.decoder(z)                  # [B,784]
recon = F.binary_cross_entropy_with_logits(
          logits, x.flatten(1), reduction="none").sum(1)
kl = 0.5 * (mu**2 + logvar.exp() - 1 - logvar).sum(1)
loss = (recon + kl).mean()                 # per image, then batch
opt.zero_grad(); loss.backward(); opt.step()
```

Encode, reparameterise, decode.

ONE TRAINING STEP

The whole algorithm fits on a slide

```
mu, logvar = model.encoder(x)              # [B,2], [B,2]
z = mu + torch.exp(0.5*logvar) * torch.randn_like(mu)
logits = model.decoder(z)                  # [B,784]
recon = F.binary_cross_entropy_with_logits(
          logits, x.flatten(1), reduction="none").sum(1)
kl = 0.5 * (mu**2 + logvar.exp() - 1 - logvar).sum(1)
loss = (recon + kl).mean()                 # per image, then batch
opt.zero_grad(); loss.backward(); opt.step()
```

Sum the BCE over all 784 pixels of each image.

ONE TRAINING STEP

The whole algorithm fits on a slide

```
mu, logvar = model.encoder(x)              # [B,2], [B,2]
z = mu + torch.exp(0.5*logvar) * torch.randn_like(mu)
logits = model.decoder(z)                  # [B,784]
recon = F.binary_cross_entropy_with_logits(
          logits, x.flatten(1), reduction="none").sum(1)
kl = 0.5 * (mu**2 + logvar.exp() - 1 - logvar).sum(1)
loss = (recon + kl).mean()                 # per image, then batch
opt.zero_grad(); loss.backward(); opt.step()
```

Sum the KL over the 2 code coordinates of each image.

ONE TRAINING STEP

The whole algorithm fits on a slide

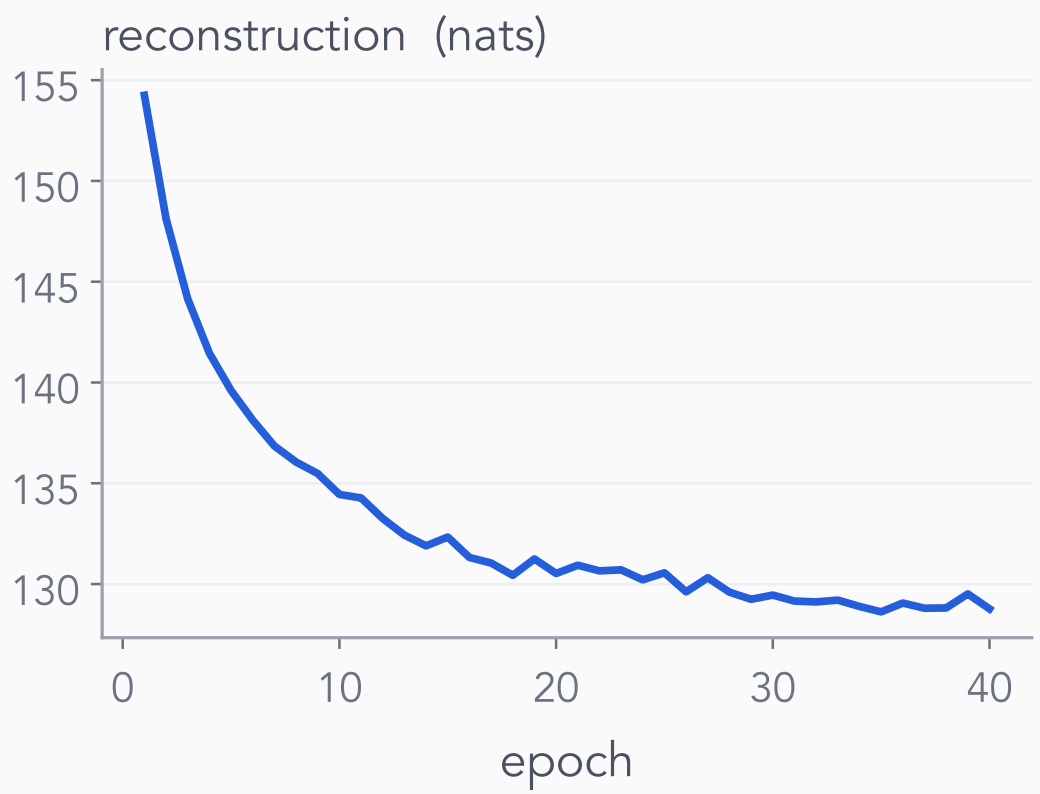
```
mu, logvar = model.encoder(x)              # [B,2], [B,2]
z = mu + torch.exp(0.5*logvar) * torch.randn_like(mu)
logits = model.decoder(z)                  # [B,784]
recon = F.binary_cross_entropy_with_logits(
          logits, x.flatten(1), reduction="none").sum(1)
kl = 0.5 * (mu**2 + logvar.exp() - 1 - logvar).sum(1)
loss = (recon + kl).mean()                 # per image, then batch
opt.zero_grad(); loss.backward(); opt.step()
```

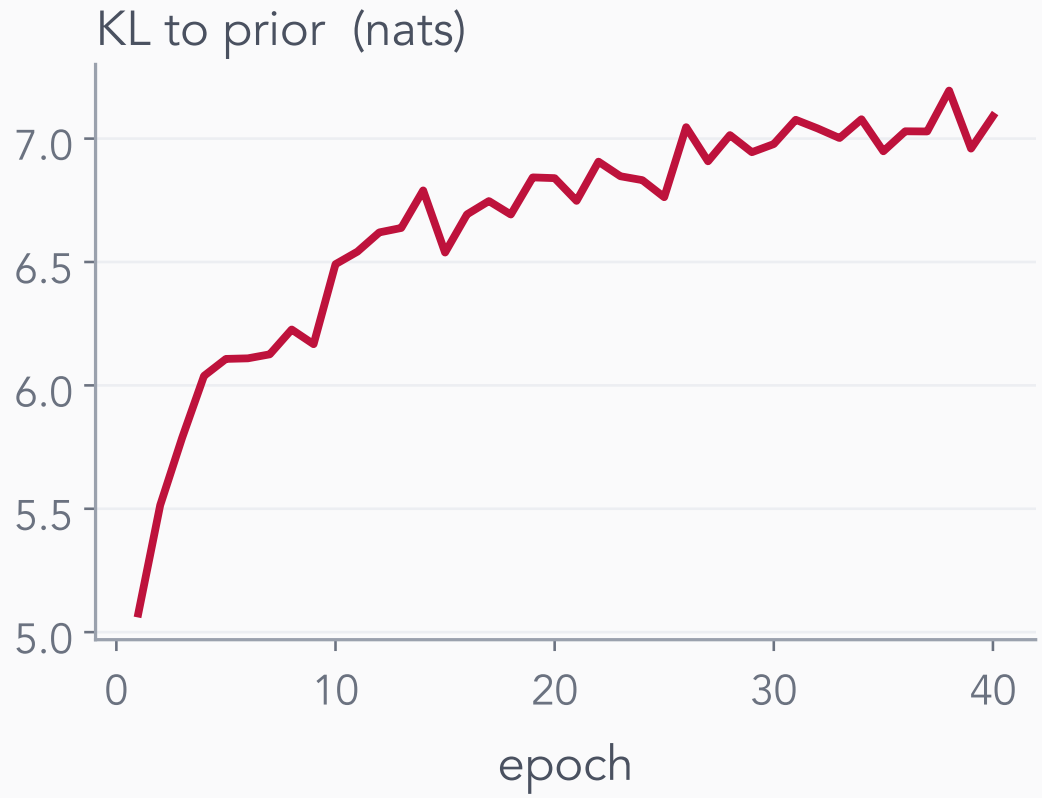
Add per image, average over the batch, step both encoder and decoder.

Averaging BCE over pixels instead of summing silently makes the KL 784× stronger.

TRAINING RUN, TEST SET

Reconstruction falls, KL rises, then both settle

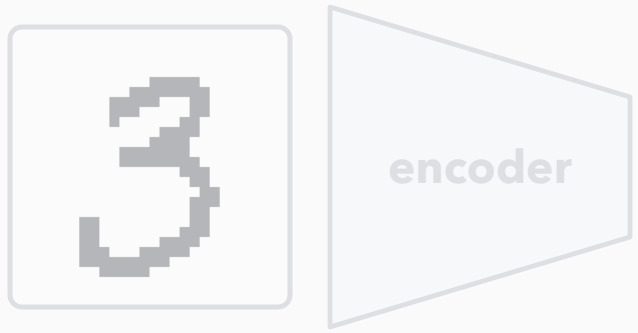




After 40 epochs: reconstruction 128.8 + KL 7.1 = 135.9 nats per test image.

THE PAYOFF

Generate: random code → decoder. No encoder.

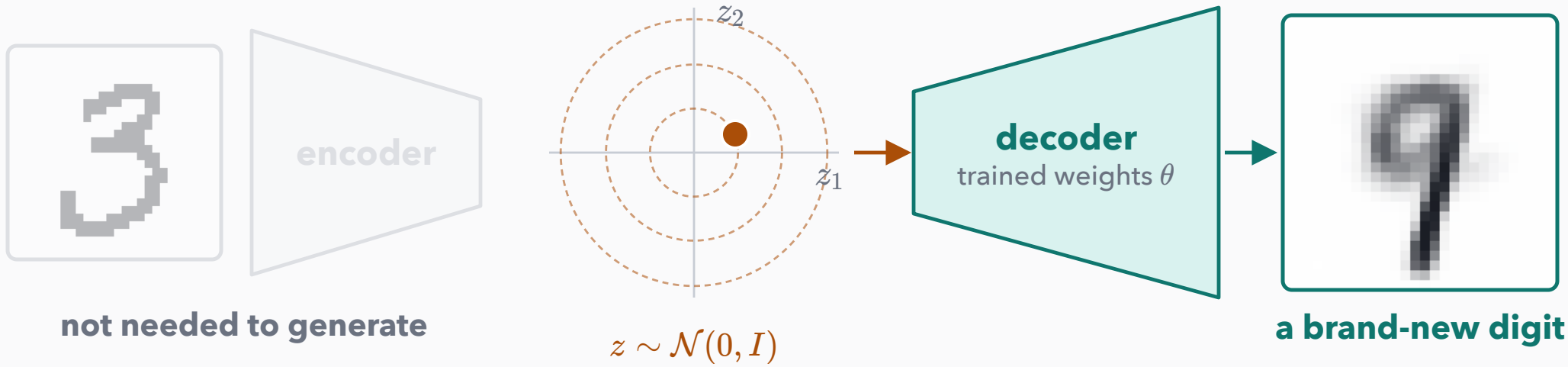


not needed to generate

> This is the whole generator. The encoder was scaffolding for training.

THE PAYOFF

Generate: random code → decoder. No encoder.



This is the whole generator. The encoder was scaffolding for training.

48 RANDOM CODES EACH

Samples from two VAEs



2-number code · reconstruction 128.8

48 RANDOM CODES EACH

Samples from two VAEs



2-number code · reconstruction 128.8

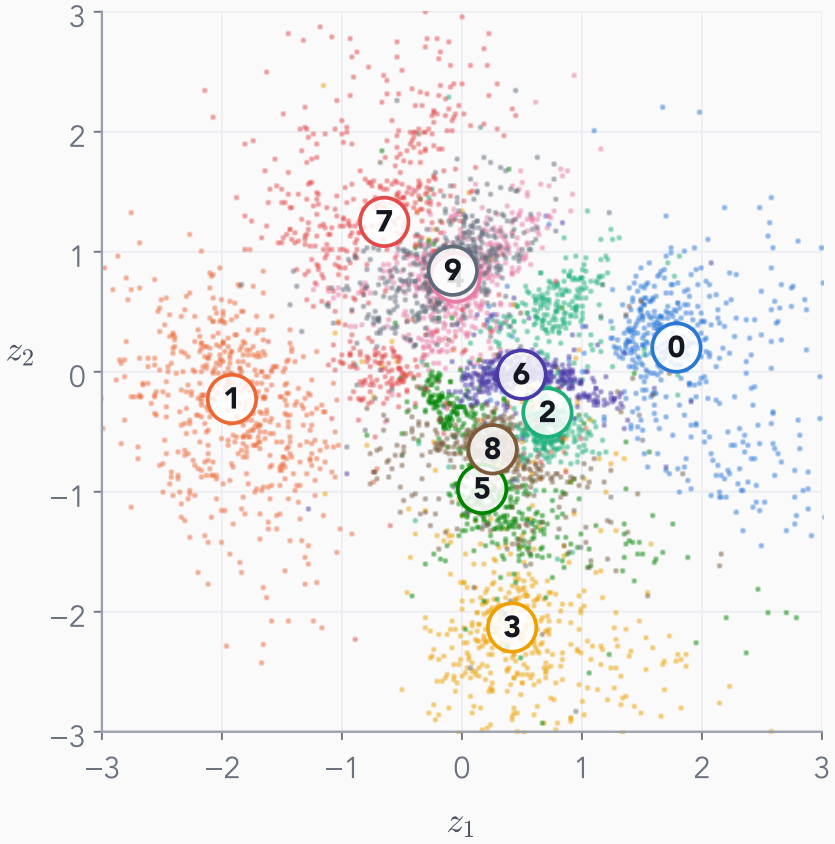


16-number code · reconstruction 57.8

More code dimensions: sharper and more varied, but some samples are odd. Neither is perfect.

BECAUSE THE CODE IS 2-D, WE CAN DRAW THE WHOLE MAP

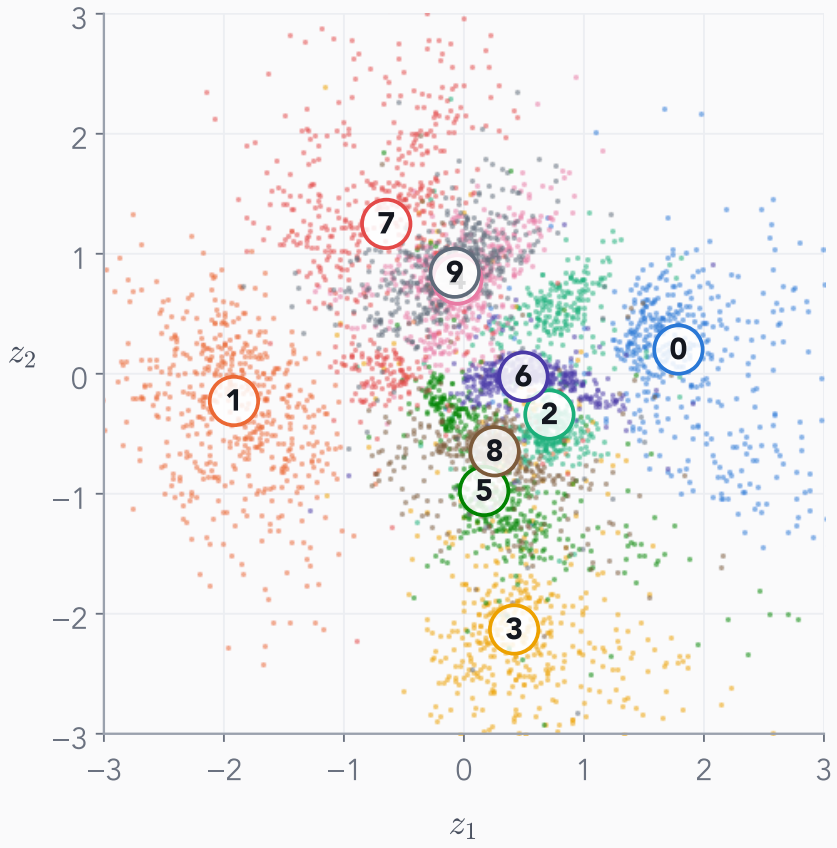
Every point in the code space is a digit



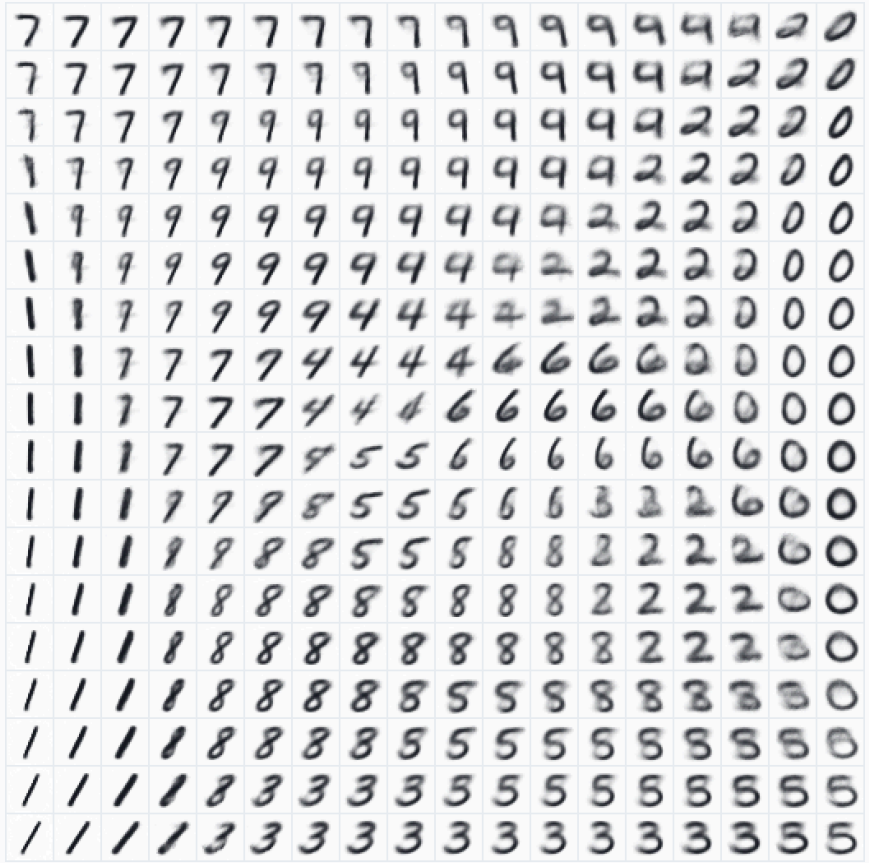
where test digits land

BECAUSE THE CODE IS 2-D, WE CAN DRAW THE WHOLE MAP

Every point in the code space is a digit



where test digits land



decoded on an 18 × 18 grid, spaced by quantiles of $\mathcal{N}(0, 1)$

Neighbouring codes give similar digits, and every region decodes to something plausible.

THE COVER, EXPLAINED

Walk in a straight line from our 3 to a 7



encode both, take 12 evenly spaced points between their μ's, decode each (16-number VAE)

THE COVER, EXPLAINED

Walk in a straight line from our 3 to a 7



encode both, take 12 evenly spaced points between their μ's, decode each (16-number VAE)

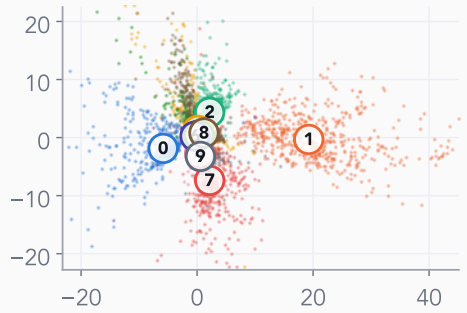


the same walk in the 2-number VAE: the line passes through the 5, 8, 4 and 9 regions on the map

THE TUG OF WAR, MADE VISIBLE

No KL, normal KL, too much KL

autoencoder (no KL)



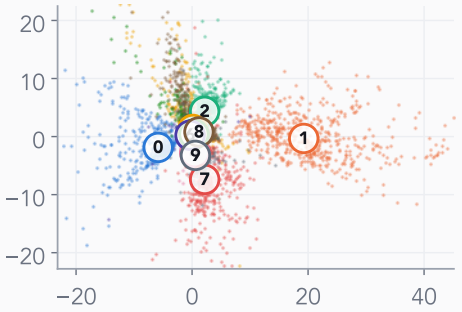


recon 128.1 · KL 0

THE TUG OF WAR, MADE VISIBLE

No KL, normal KL, too much KL

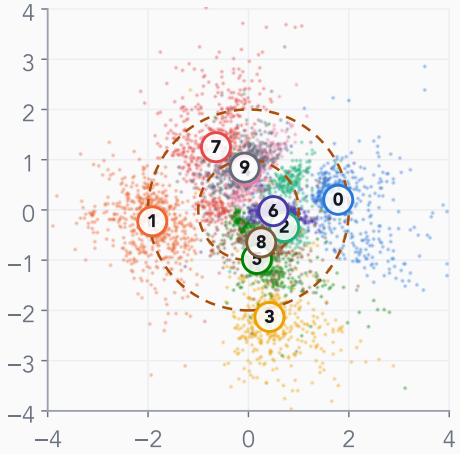
autoencoder (no KL)

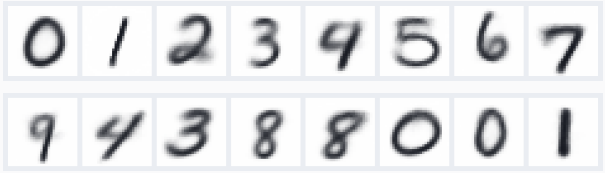




recon 128.1 · KL 0

VAE, loss = recon + KL



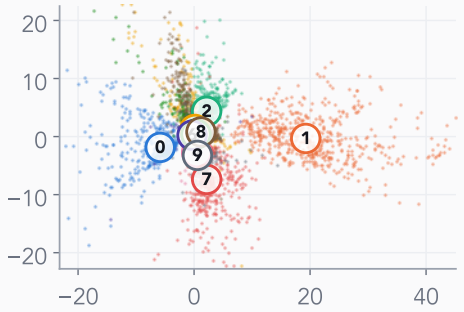


recon 128.8 · KL 7.1

THE TUG OF WAR, MADE VISIBLE

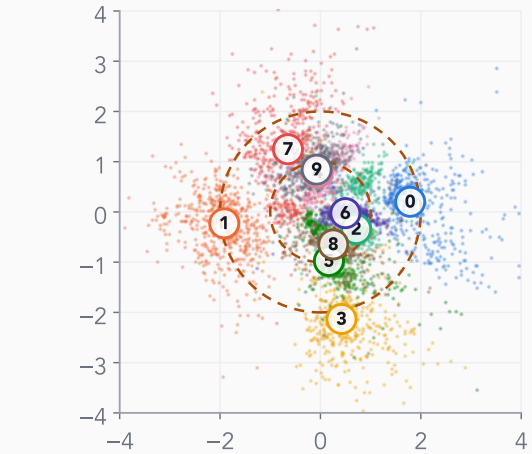
No KL, normal KL, too much KL

autoencoder (no KL)



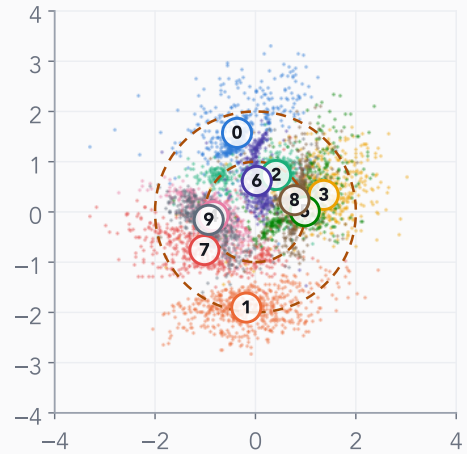
recon 128.1 · KL 0

VAE, loss = recon + KL



recon 128.8 · KL 7.1

recon + 8 × KL



recon 140.4 · KL 3.9

> **Rows: code space · reconstructions · random samples. Too little KL: samples fail. Too much: everything blurs together.**

AN HONEST LOOK

Why are VAE samples blurry?

1. It fits $\mathrm{KL}(\text{data} \,\|\, \text{model})$. Like the wide Gaussian on our bumps, it must cover every training digit, so when unsure it spreads probability.



AN HONEST LOOK

Why are VAE samples blurry?

1. It fits $\mathrm{KL}(\text{data} \,\|\, \text{model})$. Like the wide Gaussian on our bumps, it must cover every training digit, so when unsure it spreads probability.

2. Pixels are independent given z. The decoder outputs one probability per pixel. If a stroke could be here or one pixel over, the best π is grey in both places.



AN HONEST LOOK

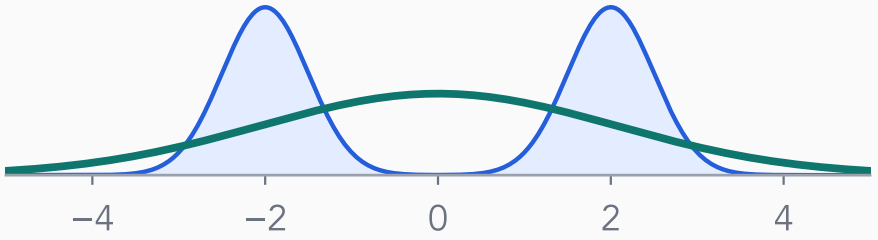
Why are VAE samples blurry?

1. It fits $\mathrm{KL}(\text{data} \,\|\, \text{model})$. Like the wide Gaussian on our bumps, it must cover every training digit, so when unsure it spreads probability.

2. Pixels are independent given z. The decoder outputs one probability per pixel. If a stroke could be here or one pixel over, the best π is grey in both places.

3. We show π, the mean, not a sample. A sample would be speckled instead of grey.

best $q = \mathcal{N}(0.0,\ 2.06^2)$





π (the mean)



a sample

Not a law of nature: bigger codes, better decoders and better likelihoods all sharpen VAEs.

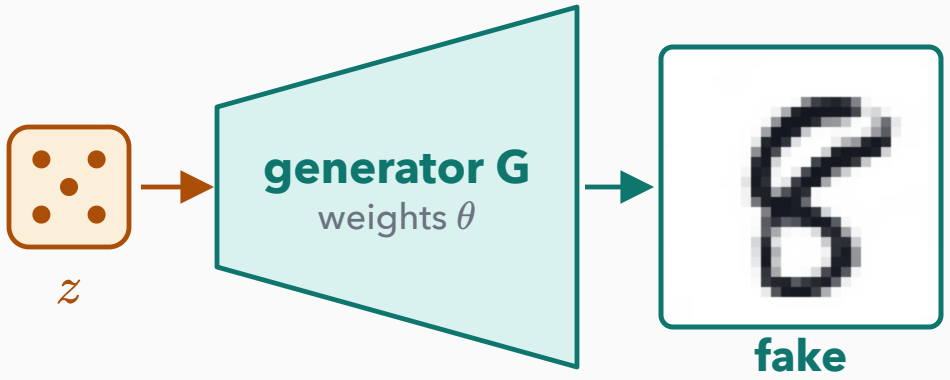
PART 7

GANs: learn the judge instead of the likelihood

Drop the encoder and the ELBO. Train a second network to tell real from fake, and train the generator to fool it.

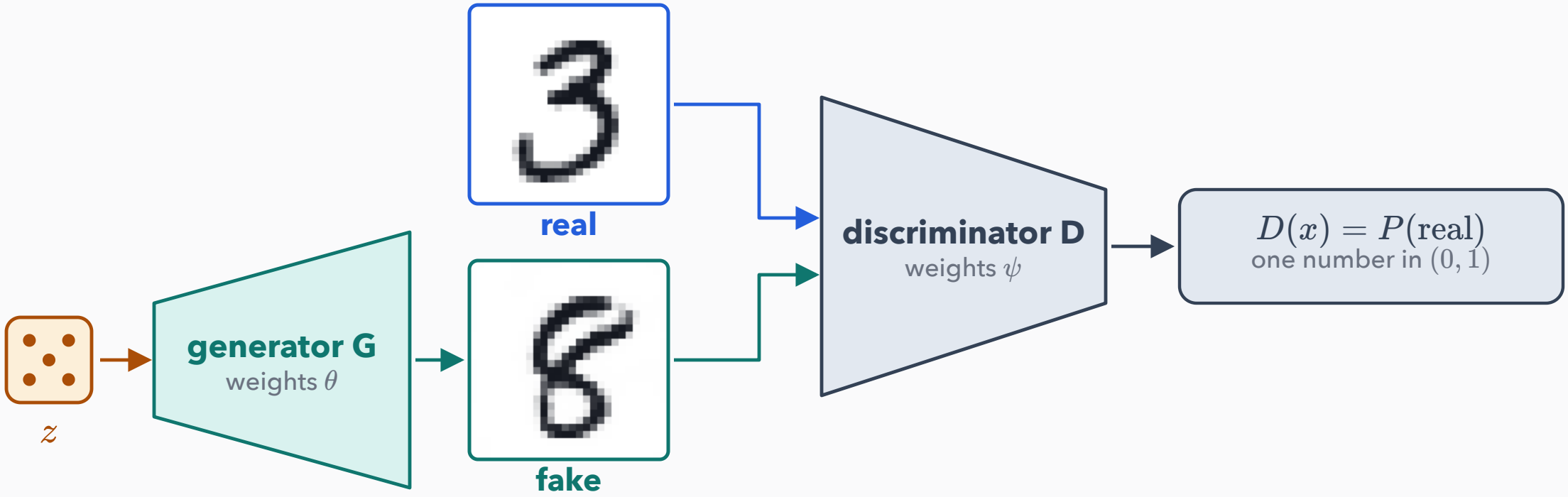
REMEMBER THE VOTE? YOU WERE THE DISCRIMINATOR.

Two networks, one game



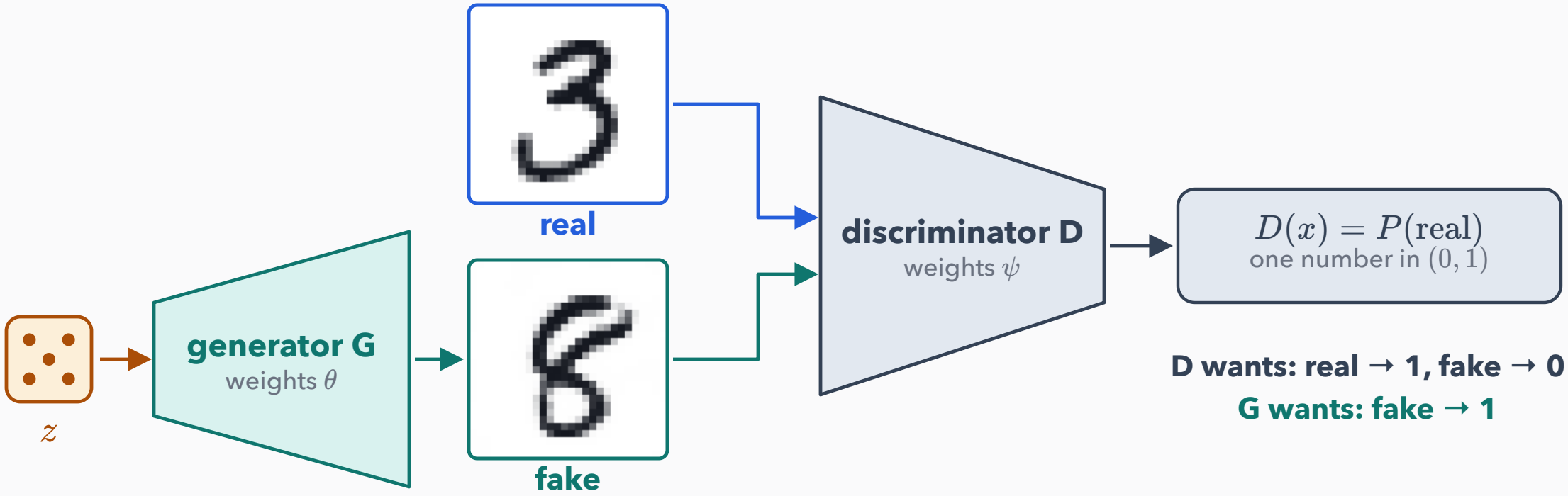
REMEMBER THE VOTE? YOU WERE THE DISCRIMINATOR.

Two networks, one game



REMEMBER THE VOTE? YOU WERE THE DISCRIMINATOR.

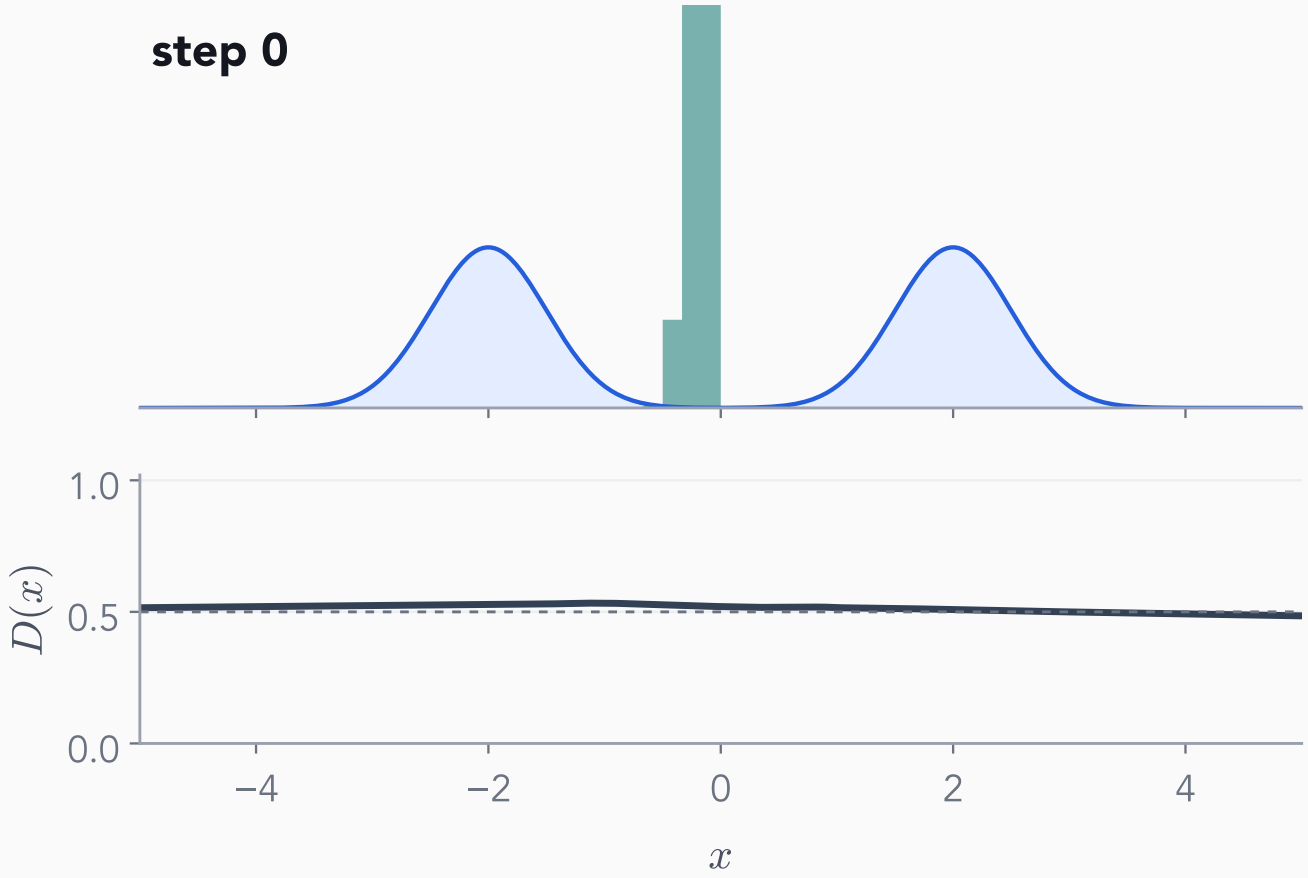
Two networks, one game



No likelihood, no encoder. The training signal for G comes entirely from D.

BACK TO OUR 1-D BUMPS

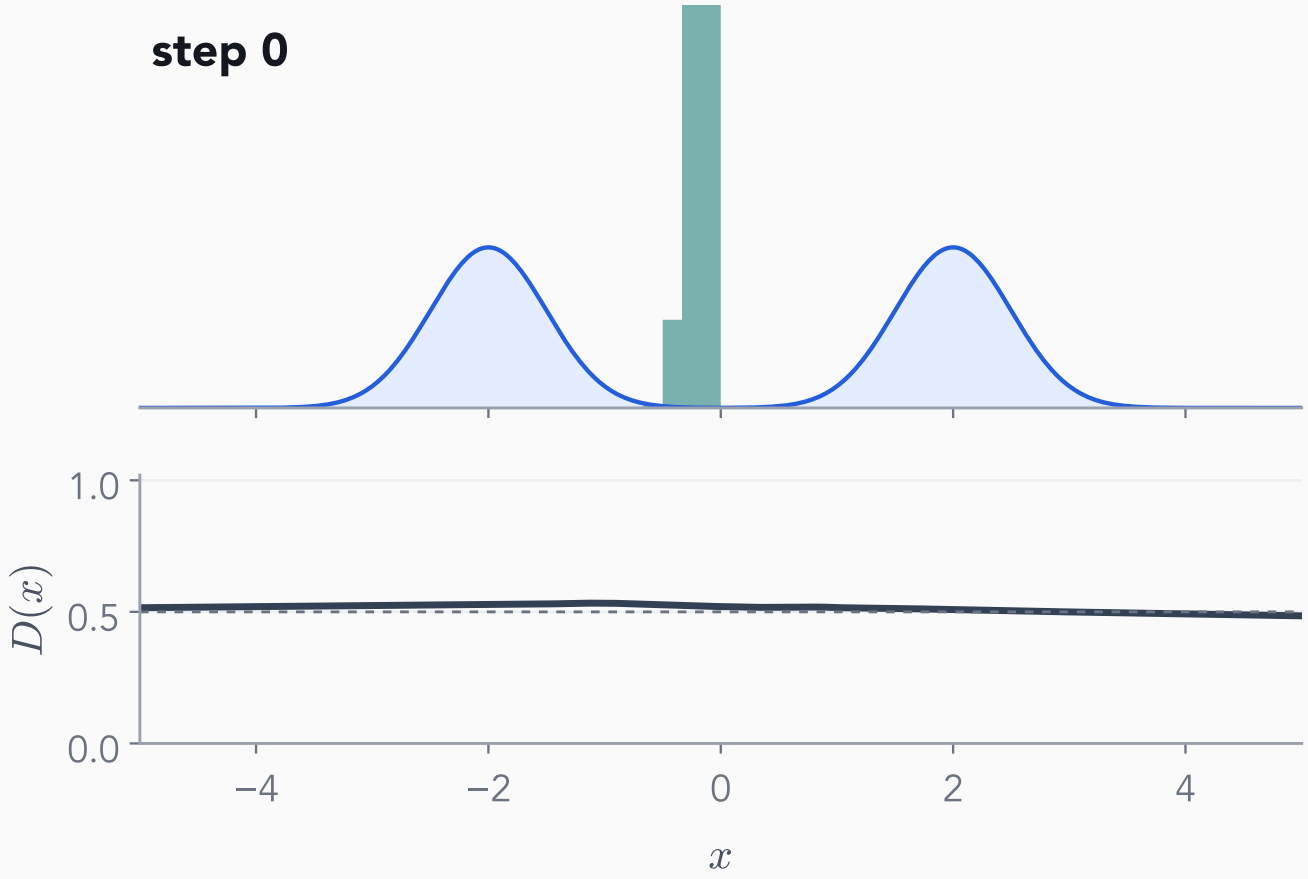
A GAN on the toy data, at step 0



- blue: the real data
- teal: G's samples for 2,000 fixed z
- slate: D(x), the probability that x is real

BACK TO OUR 1-D BUMPS

A GAN on the toy data, at step 0



- blue: the real data
- teal: G's samples for 2,000 fixed z
- slate: D(x), the probability that x is real

G: 1 number in, 1 number out. D: 1 number in, 1 logit out. Both small MLPs.

Untrained, G squeezes everything into one spike near 0 and D says 0.5 everywhere.

STEP 1 OF EACH ITERATION

The discriminator is an ordinary binary classifier

$$\mathcal{L}_D = -\log D(x_{\text{real}}) \; - \; \log\left(1 - D(G(z))\right)$$

input	label	D says	loss

STEP 1 OF EACH ITERATION

The discriminator is an ordinary binary classifier

$$\mathcal{L}_D = -\log D(x_{\text{real}}) \; - \; \log\left(1 - D(G(z))\right)$$

input	label	D says	loss
a real digit	1	0.8	$-\log 0.8 = 0.223$

STEP 1 OF EACH ITERATION

The discriminator is an ordinary binary classifier

$$\mathcal{L}_D = -\log D(x_{\text{real}}) \; - \; \log\big(1 - D(G(z))\big)$$

input	label	D says	loss
a real digit	1	0.8	$-\log 0.8 = 0.223$
a fake digit	0	0.3	$-\log(1 - 0.3) = 0.357$

STEP 1 OF EACH ITERATION

The discriminator is an ordinary binary classifier

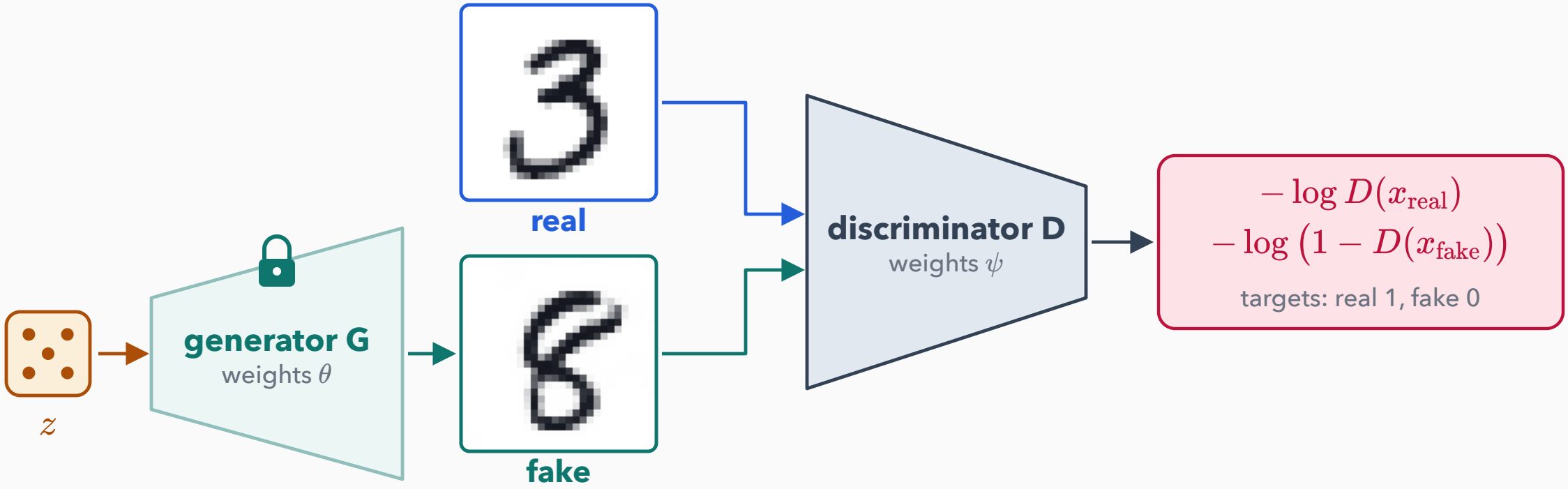
$$\mathcal{L}_D = -\log D(x_{\text{real}}) \; - \; \log\big(1 - D(G(z))\big)$$

input	label	D says	loss
a real digit	1	0.8	$-\log 0.8 = 0.223$
a fake digit	0	0.3	$-\log(1 - 0.3) = 0.357$
$\mathcal{L}_D$			**0.580**

> **Real → 1, fake → 0, binary cross-entropy. Nothing new, except that the "fake" examples keep changing.**

UPDATE D, HOLD G FIXED

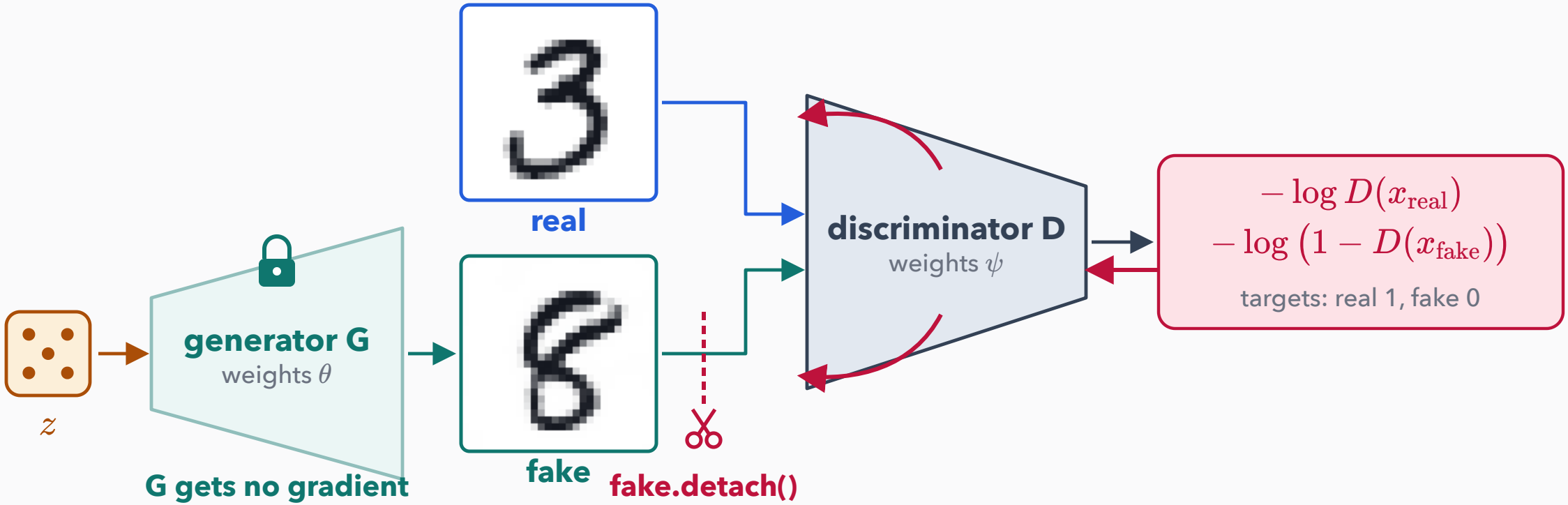
D step: cut the wire to G with detach()




```
fake = G(torch.randn(B, zdim))
loss_D = bce(D(real), ones) + bce(D(fake.detach()), zeros)
opt_D.zero_grad(); loss_D.backward(); opt_D.step()
```

UPDATE D, HOLD G FIXED

D step: cut the wire to G with detach()




```
fake = G(torch.randn(B, zdim))
loss_D = bce(D(real), ones) + bce(D(fake.detach()), zeros)
opt_D.zero_grad(); loss_D.backward(); opt_D.step()
```

WHERE G'S GRADIENT COMES FROM

The chain rule runs through D

$$\frac{\partial \mathcal{L}_G}{\partial \theta} = \underbrace{\frac{\partial \mathcal{L}_G}{\partial D}}_{= -1/D} \cdot \underbrace{\frac{\partial D}{\partial x}}_{\text{slope of } D \text{ at the fake}} \cdot \underbrace{\frac{\partial x}{\partial \theta}}_{\text{how } \theta \text{ moves the fake}}, \qquad x = G_\theta(z)$$

WHERE G'S GRADIENT COMES FROM

The chain rule runs through D

$$\frac{\partial \mathcal{L}_G}{\partial \theta} = \underbrace{\frac{\partial \mathcal{L}_G}{\partial D}}_{= -1/D} \cdot \underbrace{\frac{\partial D}{\partial x}}_{\text{slope of } D \text{ at the fake}} \cdot \underbrace{\frac{\partial x}{\partial \theta}}_{\text{how } \theta \text{ moves the fake}}, \qquad x = G_\theta(z)$$

$-1/D$ is always negative:
raising D always lowers $\mathcal{L}_G$.

WHERE G'S GRADIENT COMES FROM

The chain rule runs through D

$$\frac{\partial \mathcal{L}_G}{\partial \theta} = \underbrace{\frac{\partial \mathcal{L}_G}{\partial D}}_{= -1/D} \cdot \underbrace{\frac{\partial D}{\partial x}}_{\text{slope of } D \text{ at the fake}} \cdot \underbrace{\frac{\partial x}{\partial \theta}}_{\text{how } \theta \text{ moves the fake}}, \qquad x = G_\theta(z)$$

$-1/D$ is always negative: raising D always lowers $\mathcal{L}_G$.

$\partial D/\partial x$: which way is "more real" at this fake. Needs D's weights, but does not change them.

WHERE G'S GRADIENT COMES FROM

The chain rule runs through D

$$\frac{\partial \mathcal{L}_G}{\partial \theta} = \underbrace{\frac{\partial \mathcal{L}_G}{\partial D}}_{= -1/D} \cdot \underbrace{\frac{\partial D}{\partial x}}_{\text{slope of } D \text{ at the fake}} \cdot \underbrace{\frac{\partial x}{\partial \theta}}_{\text{how } \theta \text{ moves the fake}}, \qquad x = G_\theta(z)$$

$-1/D$ is always negative: raising D always lowers $\mathcal{L}_G$.

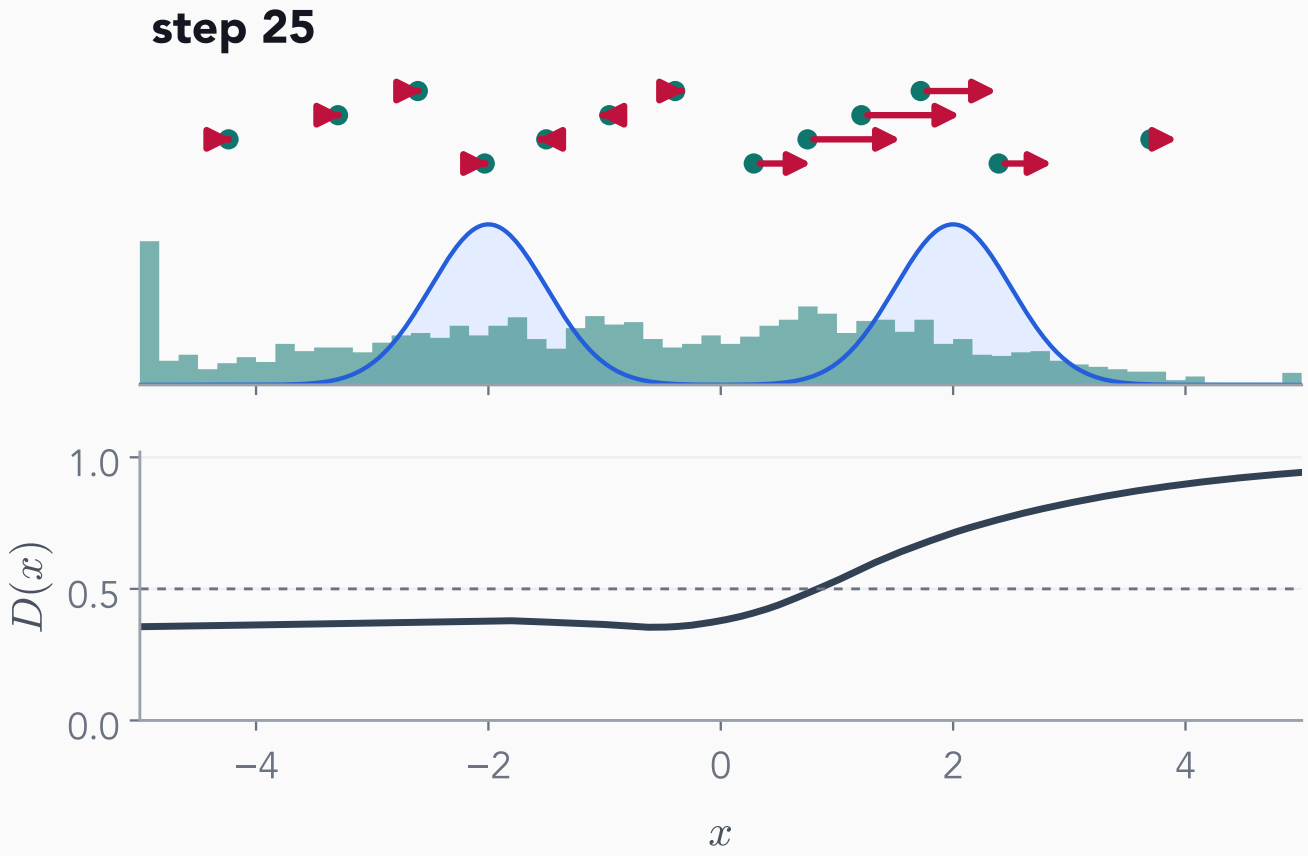
$\partial D/\partial x$: which way is "more real" at this fake. Needs D's weights, but does not change them.

$\partial x/\partial \theta$: ordinary backprop through G.

So the fake moves uphill on D, and G's weights change to make that move. Next: watch it on the 1-D data.

STEP 2: WHAT WOULD FOOL D?

G step: push every fake uphill on D(x)

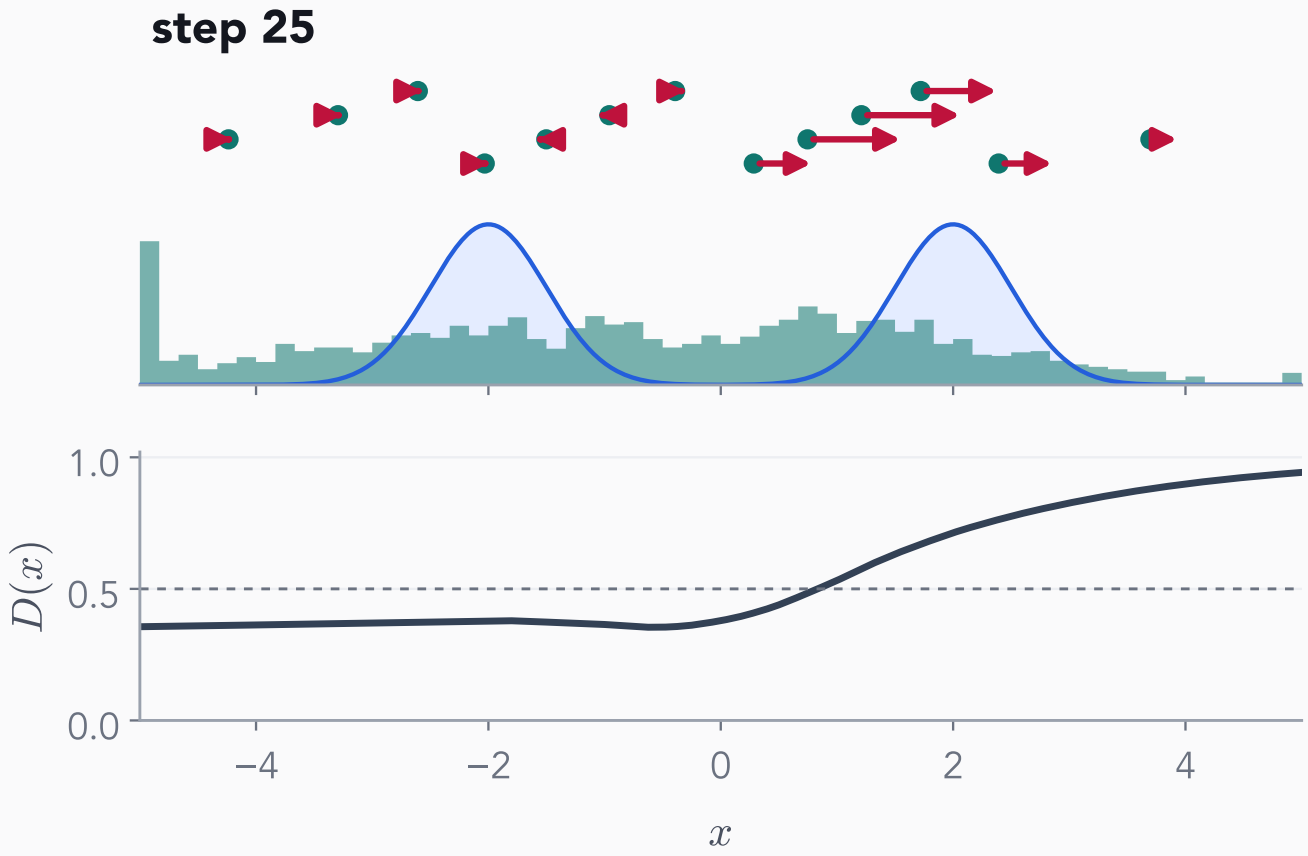


$$\mathcal{L}_G = -\log D(G(z))$$

Same BCE, but now the fakes are labelled 1: "make D call these real".

STEP 2: WHAT WOULD FOOL D?

G step: push every fake uphill on D(x)



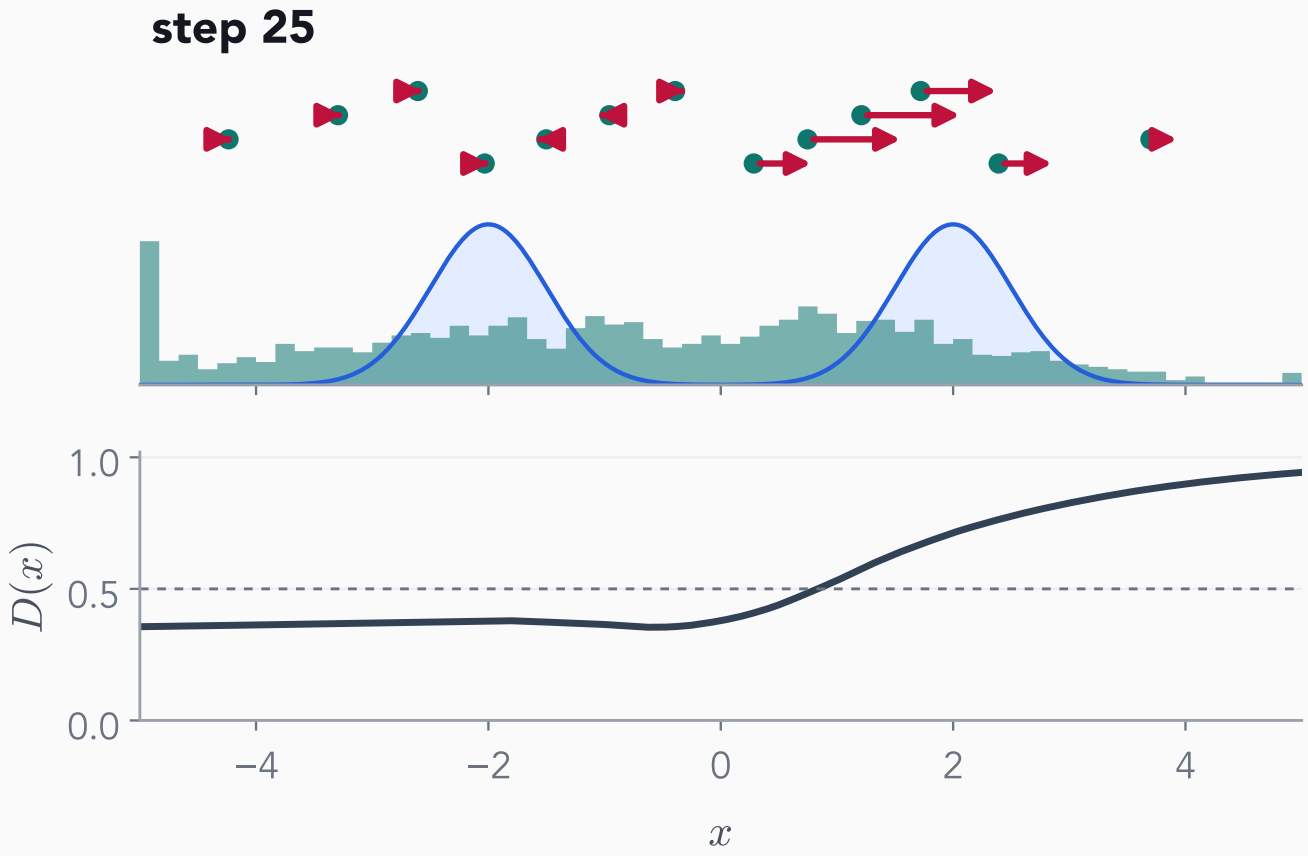
$$\mathcal{L}_G = -\log D(G(z))$$

Same BCE, but now the fakes are labelled 1: "make D call these real".

The gradient tells each fake which way D(x) increases. Red arrows: real dD/dx at step 25 of our run.

STEP 2: WHAT WOULD FOOL D?

G step: push every fake uphill on D(x)



$$\mathcal{L}_G = -\log D(G(z))$$

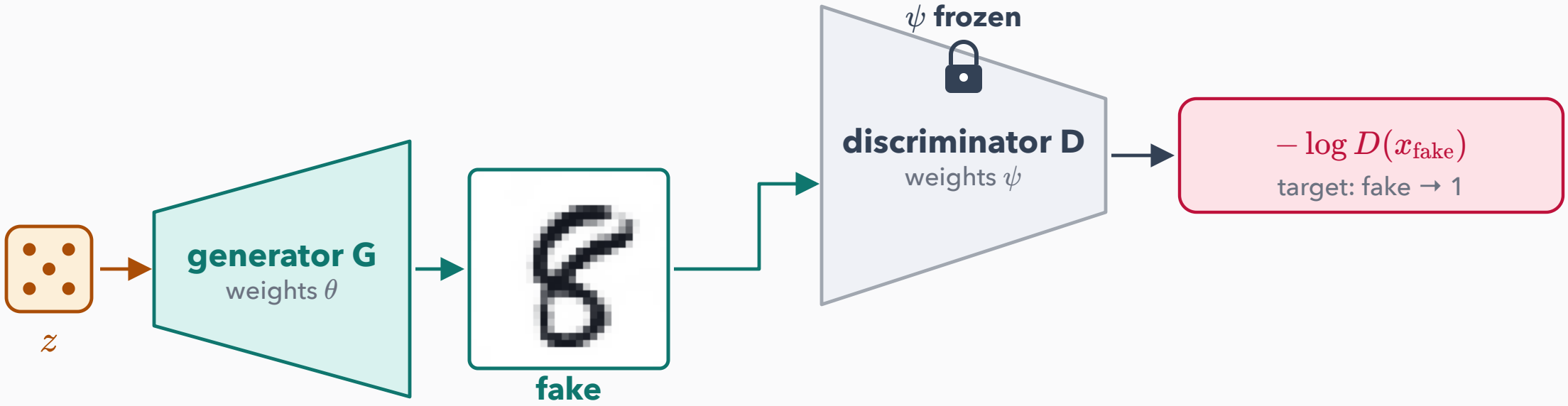
Same BCE, but now the fakes are labelled 1: "make D call these real".

The gradient tells each fake which way D(x) increases. Red arrows: real dD/dx at step 25 of our run.

G then changes its weights θ so that its outputs move that way.

UPDATE G, HOLD D FIXED

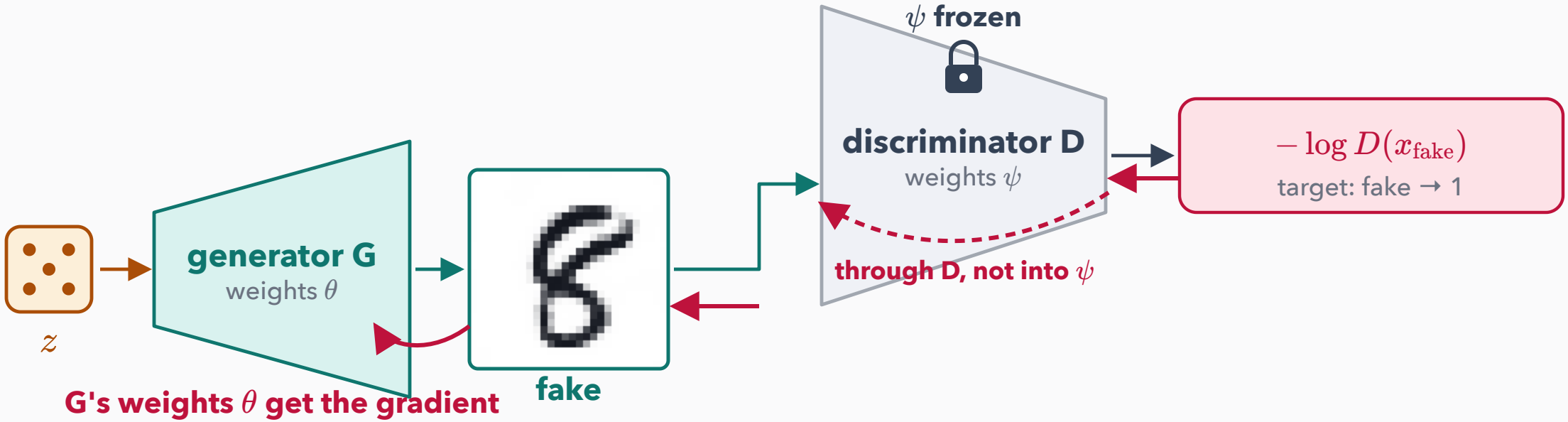
G step: lock D's weights, keep the wire




```
D.requires_grad_(False)                    # lock D's weights, keep the path
loss_G = bce(D(G(torch.randn(B, zdim))), ones)   # fresh fakes, no detach
opt_G.zero_grad(); loss_G.backward(); opt_G.step(); D.requires_grad_(True)
```

UPDATE G, HOLD D FIXED

G step: lock D's weights, keep the wire




```
D.requires_grad_(False)                    # lock D's weights, keep the path
loss_G = bce(D(G(torch.randn(B, zdim))), ones)   # fresh fakes, no detach
opt_G.zero_grad(); loss_G.backward(); opt_G.step(); D.requires_grad_(True)
```

THE MOST COMMON GAN BUG

Freezing is not detaching

detach(): cut the wire



the gradient stops at the cut:
G learns nothing

THE MOST COMMON GAN BUG

Freezing is not detaching

detach(): cut the wire



the gradient stops at the cut: G learns nothing

requires_grad_(False): lock D's weights, keep the wire

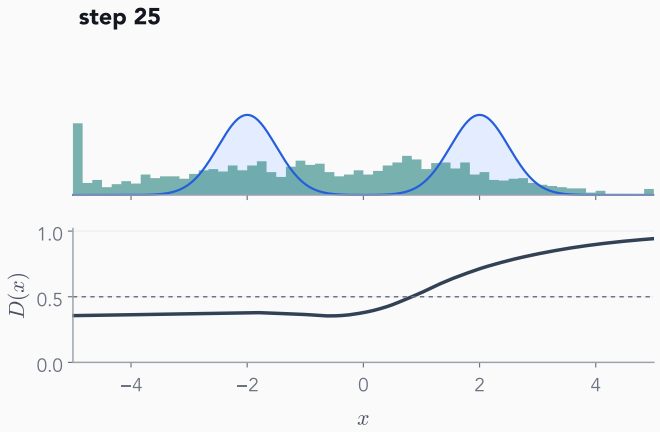


D's weights stay put; G still learns

> Detach the fake in the G step and loss_G still prints a number, but G never learns. Silent failure.

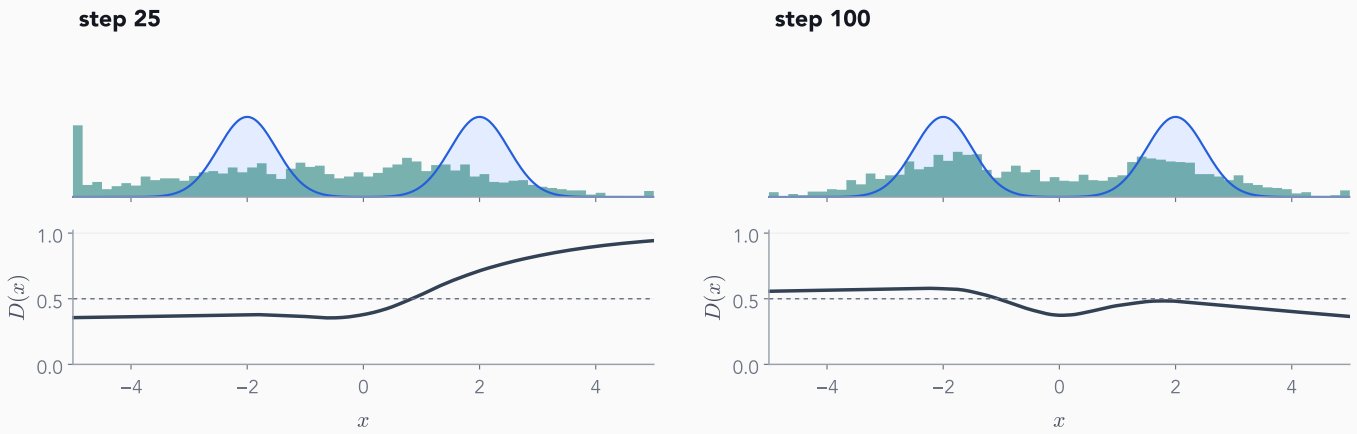
ALTERNATE THE TWO STEPS 3,000 TIMES

Watch the game on the toy data



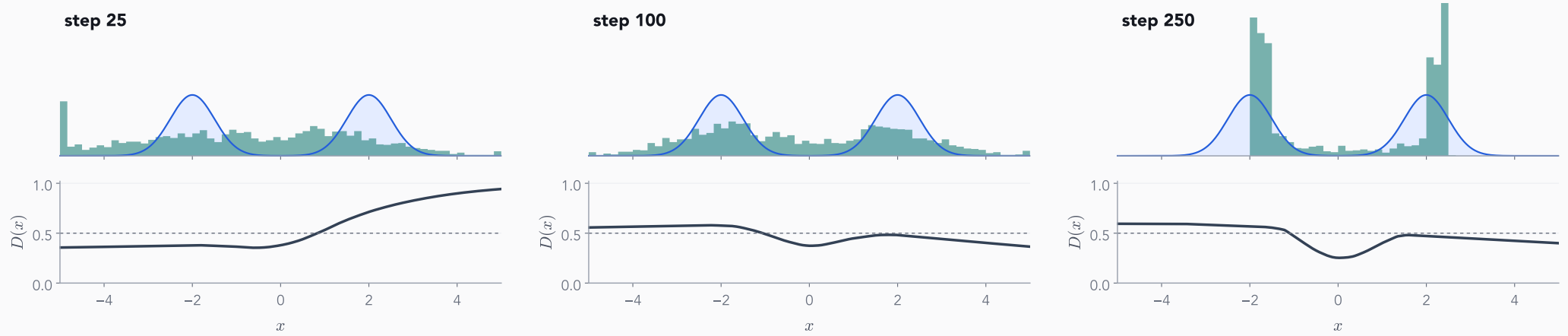
ALTERNATE THE TWO STEPS 3,000 TIMES

Watch the game on the toy data



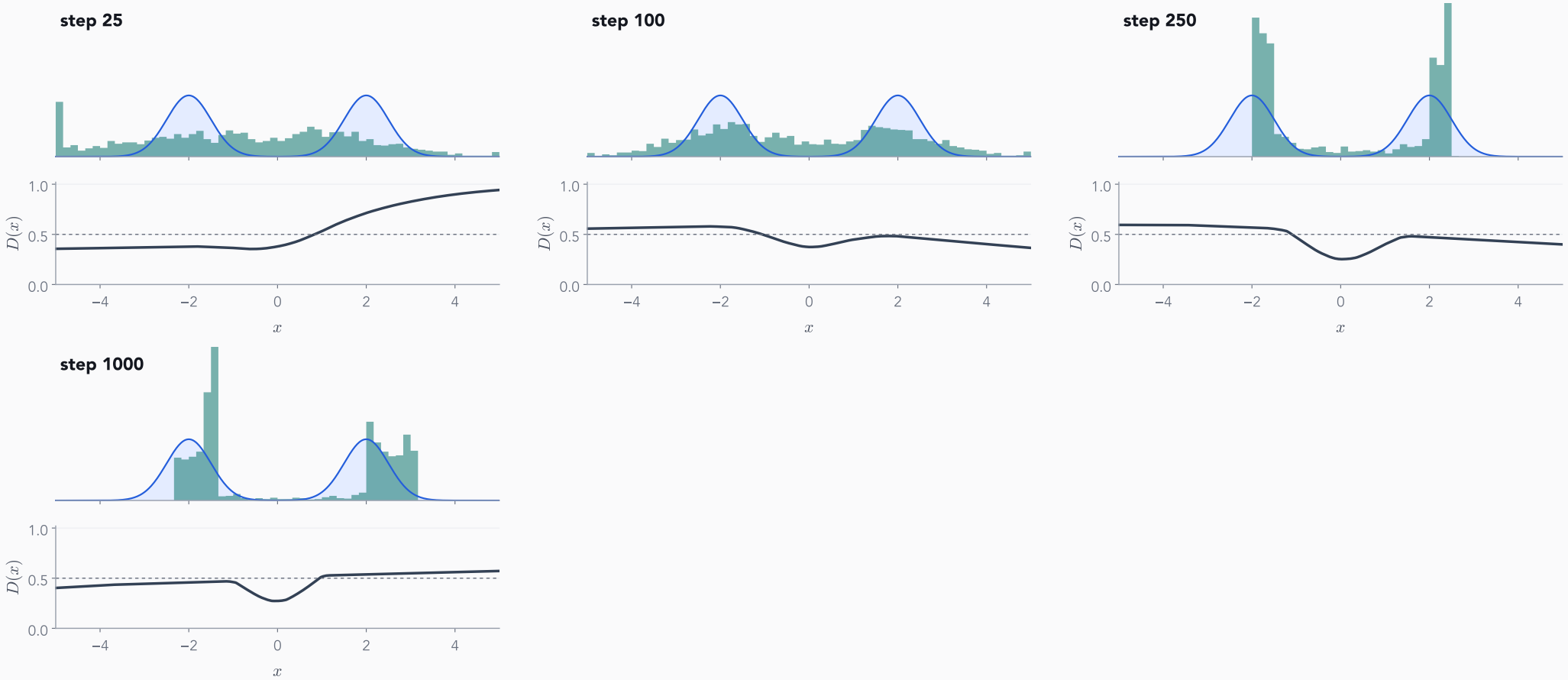
ALTERNATE THE TWO STEPS 3,000 TIMES

Watch the game on the toy data



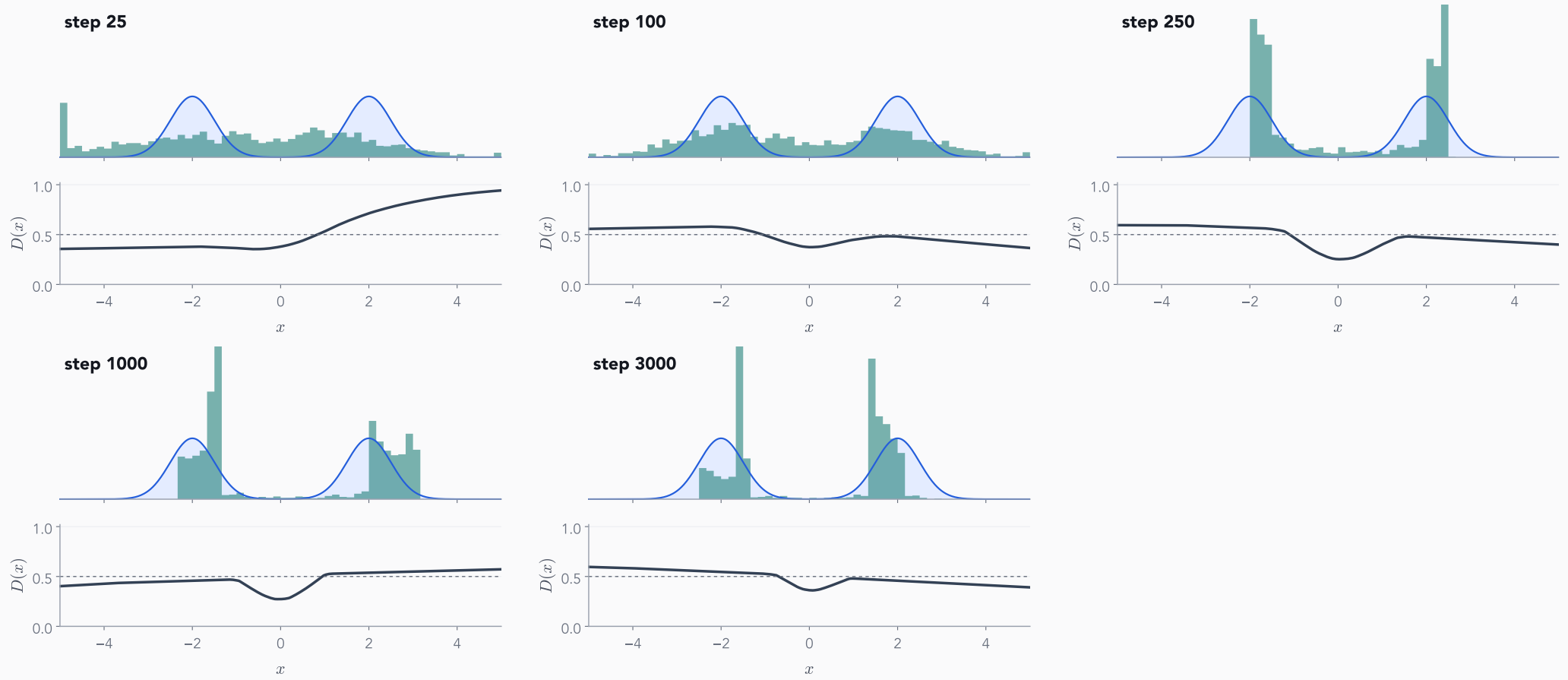
ALTERNATE THE TWO STEPS 3,000 TIMES

Watch the game on the toy data



ALTERNATE THE TWO STEPS 3,000 TIMES

Watch the game on the toy data



By step 3,000 the fakes cover both bumps, and D(x) is near 0.5 wherever the data is: it can no longer tell.

THE TEXTBOOK FORMULA

A two-player minimax game

$$\min_{G} \max_{D} \; \mathbb{E}_{x \sim p_{\text{data}}} \big[\log D(x) \big] \; + \; \mathbb{E}_{z \sim p(z)} \big[\log \big(1 - D(G(z)) \big) \big]$$

THE TEXTBOOK FORMULA

A two-player minimax game

$$\min_{G} \max_{D} \; \mathbb{E}_{x \sim p_{\text{data}}} \big[\log D(x) \big] \; + \; \mathbb{E}_{z \sim p(z)} \big[\log \big(1 - D(G(z)) \big) \big]$$

D maximises: high D on real data, low D on fakes. This is our $-\mathcal{L}_D$.

THE TEXTBOOK FORMULA

A two-player minimax game

$$\min_{G} \max_{D} \; \mathbb{E}_{x \sim p_{\text{data}}} \big[\log D(x) \big] \; + \; \mathbb{E}_{z \sim p(z)} \big[\log \big(1 - D(G(z)) \big) \big]$$

D maximises: high D on real data, low D on fakes. This is our $-\mathcal{L}_D$.

G minimises: only the second term involves G. It wants $D(G(z))$ high.

THE TEXTBOOK FORMULA

A two-player minimax game

$$\min_{G} \max_{D} \; \mathbb{E}_{x \sim p_{\text{data}}} \big[\log D(x) \big] \; + \; \mathbb{E}_{z \sim p(z)} \big[\log \big(1 - D(G(z)) \big) \big]$$

D maximises: high D on real data, low D on fakes. This is our $-\mathcal{L}_D$.

G minimises: only the second term involves G. It wants $D(G(z))$ high.

With a perfect D, G's best move is $p_G = p_{\text{data}}$ (Goodfellow et al. 2014). Real training, with finite networks and steps, has no such guarantee.

Differentiate both with respect to D's logit a

$$D = \text{sigmoid}(a), \qquad \frac{dD}{da} = D\,(1 - D)$$

Differentiate both with respect to D's logit a

$$D = \text{sigmoid}(a), \qquad \frac{dD}{da} = D\,(1 - D)$$

minimax: minimise $\log(1 - D)$

$$\frac{d}{da}\log(1 - D) = \frac{-D(1 - D)}{1 - D} = -D$$

Differentiate both with respect to D's logit a

$$D = \text{sigmoid}(a), \qquad \frac{dD}{da} = D\,(1 - D)$$

minimax: minimise $\log(1 - D)$

$$\frac{d}{da}\log(1 - D) = \frac{-D(1 - D)}{1 - D} = -D$$

non-saturating: minimise $-\log D$

$$\frac{d}{da}\big(-\log D\big) = \frac{-D(1 - D)}{D} = D - 1$$

Differentiate both with respect to D's logit a

$$D = \text{sigmoid}(a), \qquad \frac{dD}{da} = D\,(1 - D)$$

minimax: minimise $\log(1 - D)$

$$\frac{d}{da}\log(1 - D) = \frac{-D(1 - D)}{1 - D} = -D$$

$$D = 0.01 : \quad -0.01$$

non-saturating: minimise $-\log D$

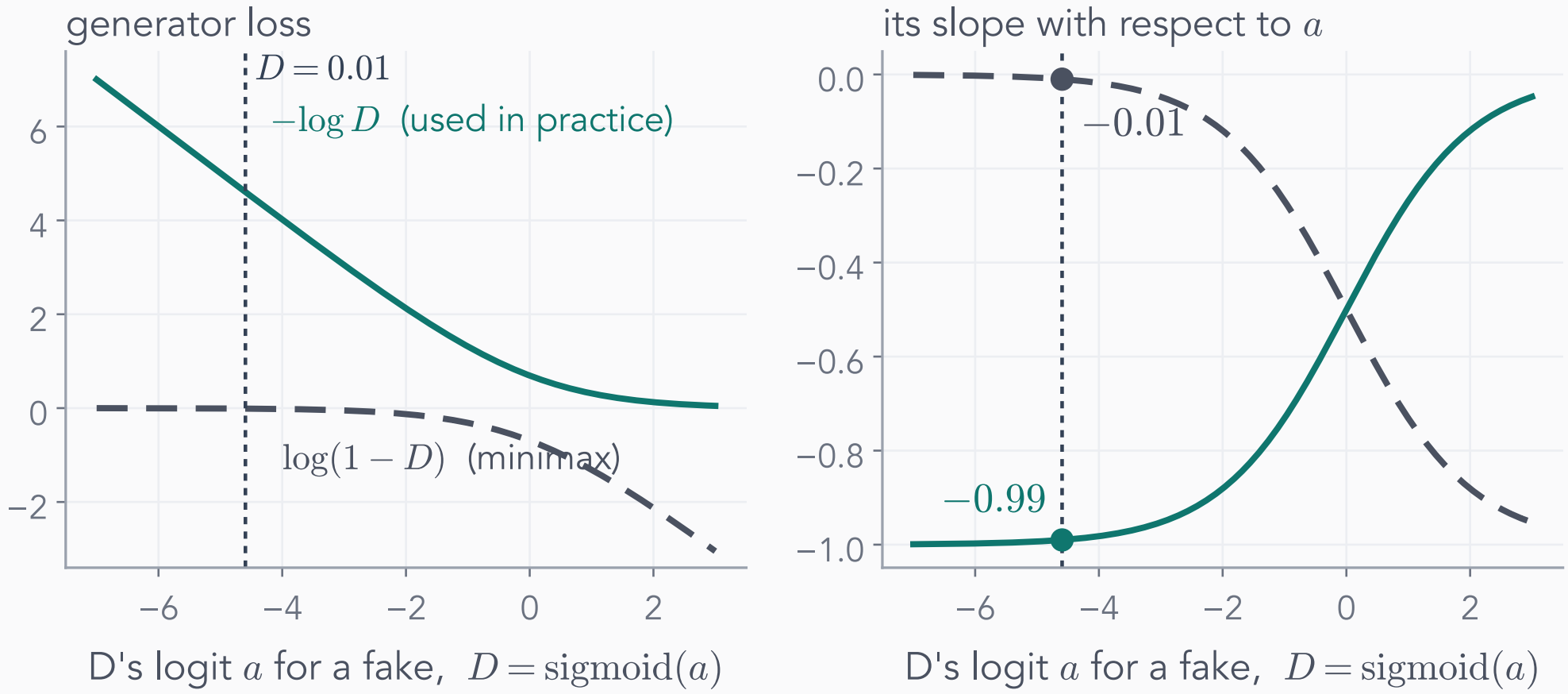
$$\frac{d}{da}\big(-\log D\big) = \frac{-D(1 - D)}{D} = D - 1$$

$$D = 0.01 : \quad -0.99$$

> **Early in training D easily rejects fakes, so $D \approx 0$: the minimax slope vanishes, the non-saturating one does not.**

WHY THE CODE USES A DIFFERENT GENERATOR LOSS

Early on, the minimax loss gives G almost no signal



When D easily spots a fake ($D = 0.01$): slope -0.01 for $\log(1 - D)$, slope -0.99 for $-\log D$. Same goal, a 99× stronger push.

THE COMPLETE UPDATE

One GAN iteration in code

```
# ---- D step: real → 1, fake → 0, G receives nothing
fake = G(torch.randn(B, zdim))
loss_D = bce_logits(D(real), ones) + bce_logits(D(fake.detach()), zeros)
opt_D.zero_grad(); loss_D.backward(); opt_D.step()

# ---- G step: fresh fakes, D's weights locked, gradient flows through D
D.requires_grad_(False)
loss_G = bce_logits(D(G(torch.randn(B, zdim))), ones)
opt_G.zero_grad(); loss_G.backward(); opt_G.step()
D.requires_grad_(True)
```

Exactly this function trained the 1-D GAN and the MNIST GAN. Only G and D differ.

THE COMPLETE UPDATE

One GAN iteration in code

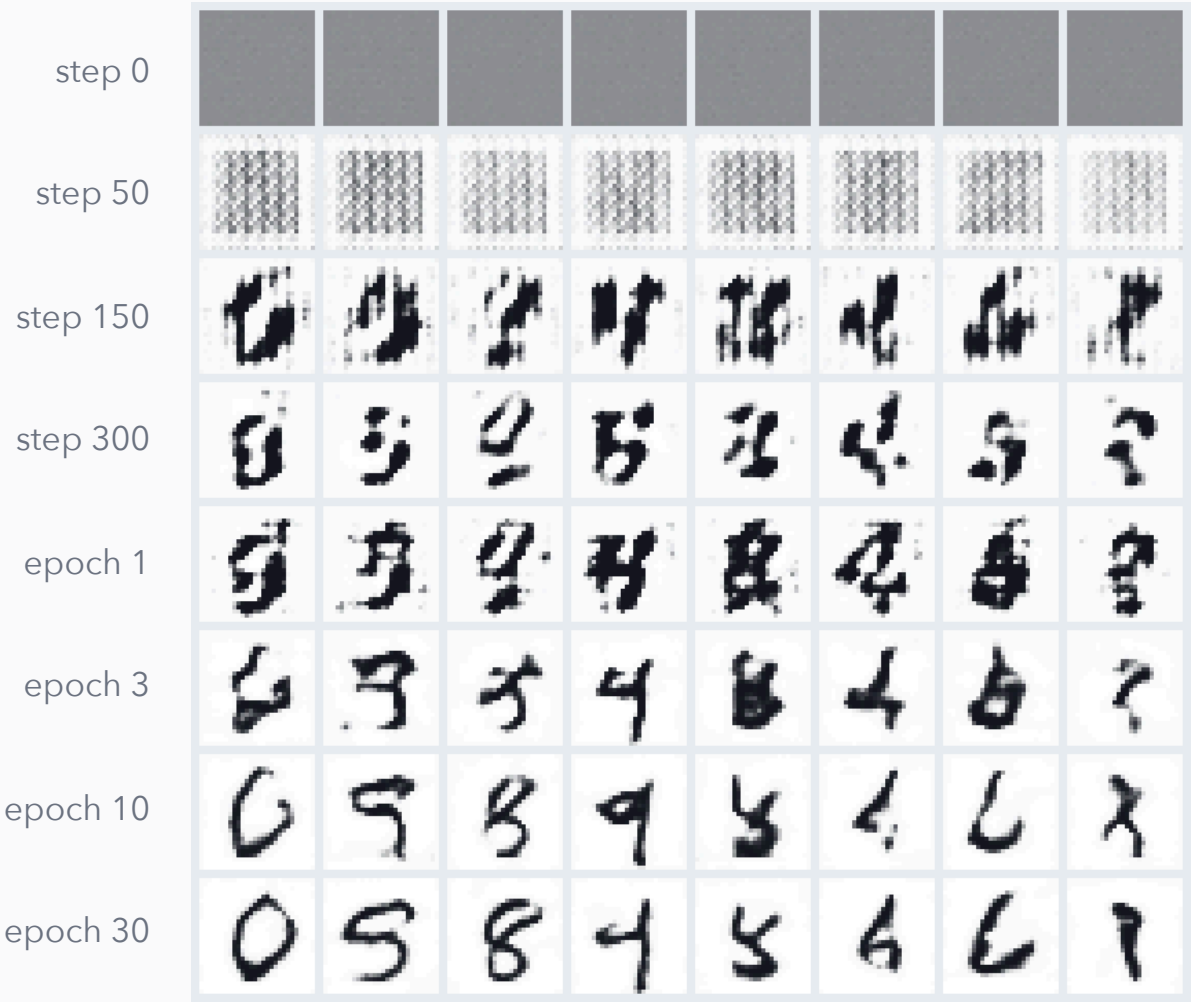
```
# ---- D step: real → 1, fake → 0, G receives nothing
fake = G(torch.randn(B, zdim))
loss_D = bce_logits(D(real), ones) + bce_logits(D(fake.detach()), zeros)
opt_D.zero_grad(); loss_D.backward(); opt_D.step()

# ---- G step: fresh fakes, D's weights locked, gradient flows through D
D.requires_grad_(False)
loss_G = bce_logits(D(G(torch.randn(B, zdim))), ones)
opt_G.zero_grad(); loss_G.backward(); opt_G.step()
D.requires_grad_(True)
```

Exactly this function trained the 1-D GAN and the MNIST GAN. Only G and D differ.

ON MNIST: THE SAME 8 CODES, THROUGH TRAINING

Watch the samples, not the loss

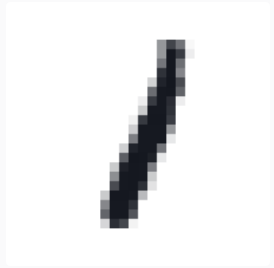


BACK TO THE VOTE

Which were real, and what did our discriminator think?



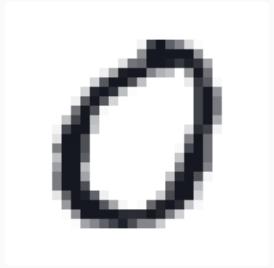
1



2



3



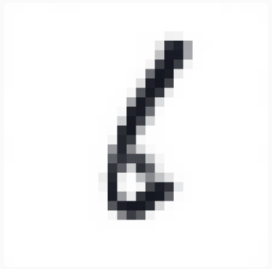
4



5



6



7



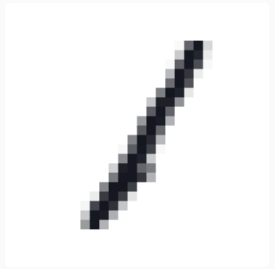
8



9



10



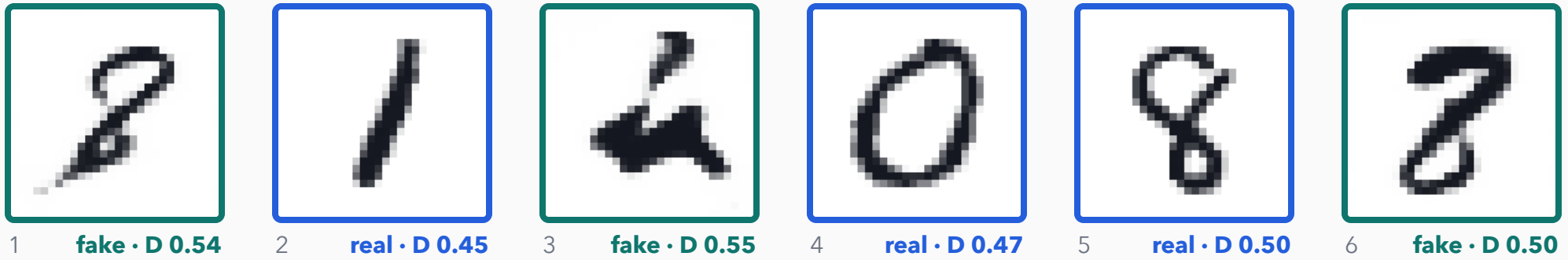
11



12

BACK TO THE VOTE

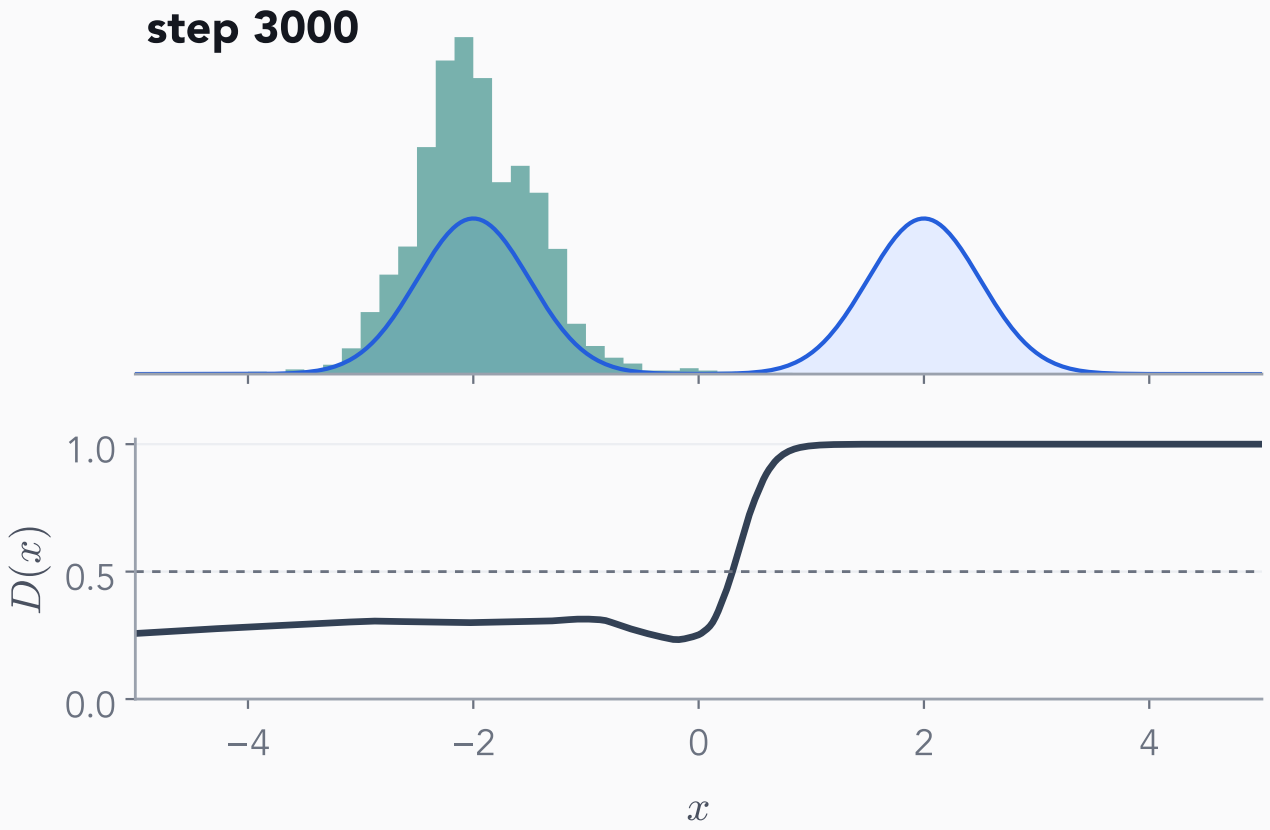
Which were real, and what did our discriminator think?



> Our trained D gives every tile between 0.39 and 0.55. At the end of training it can barely tell either.

THE CLASSIC FAILURE

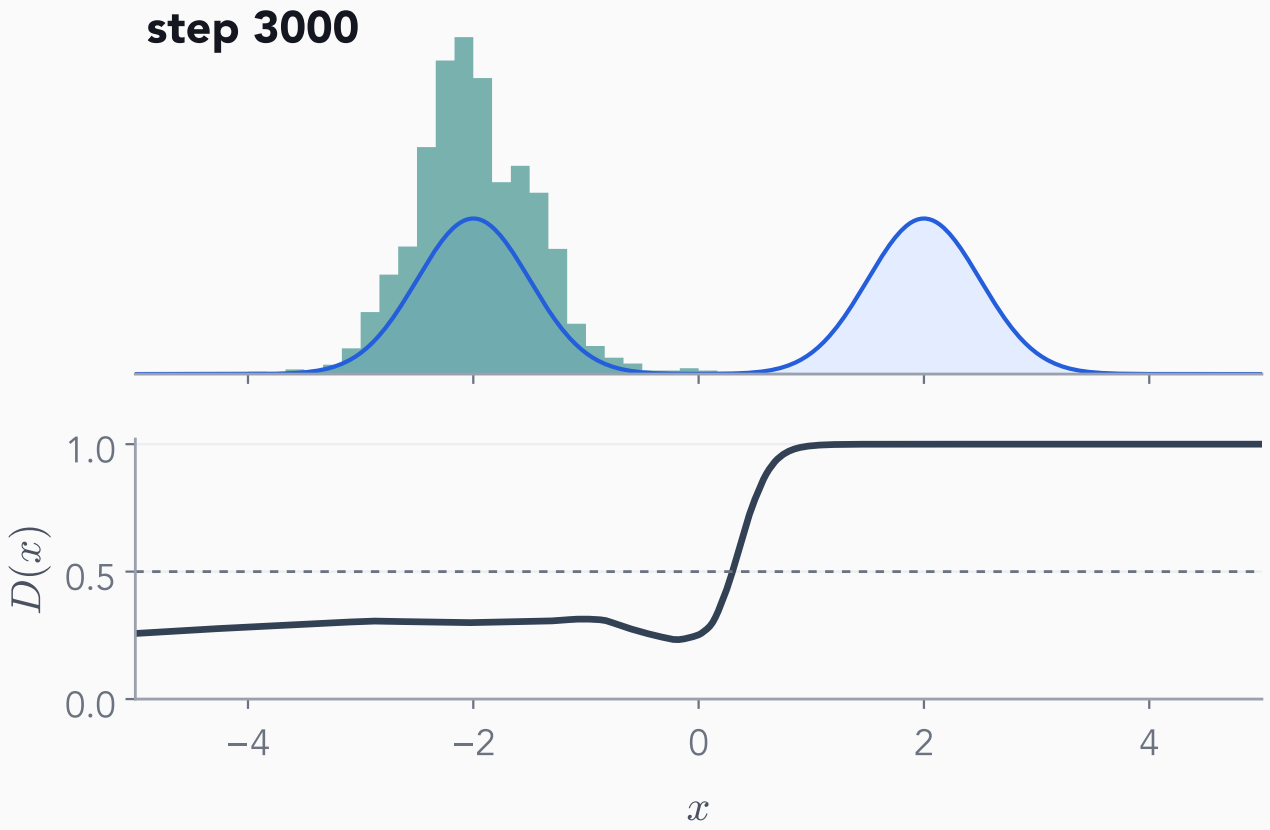
Mode collapse: perfect samples of only one kind



Same data, same training loop, but G has only 4 hidden units per layer.

THE CLASSIC FAILURE

Mode collapse: perfect samples of only one kind

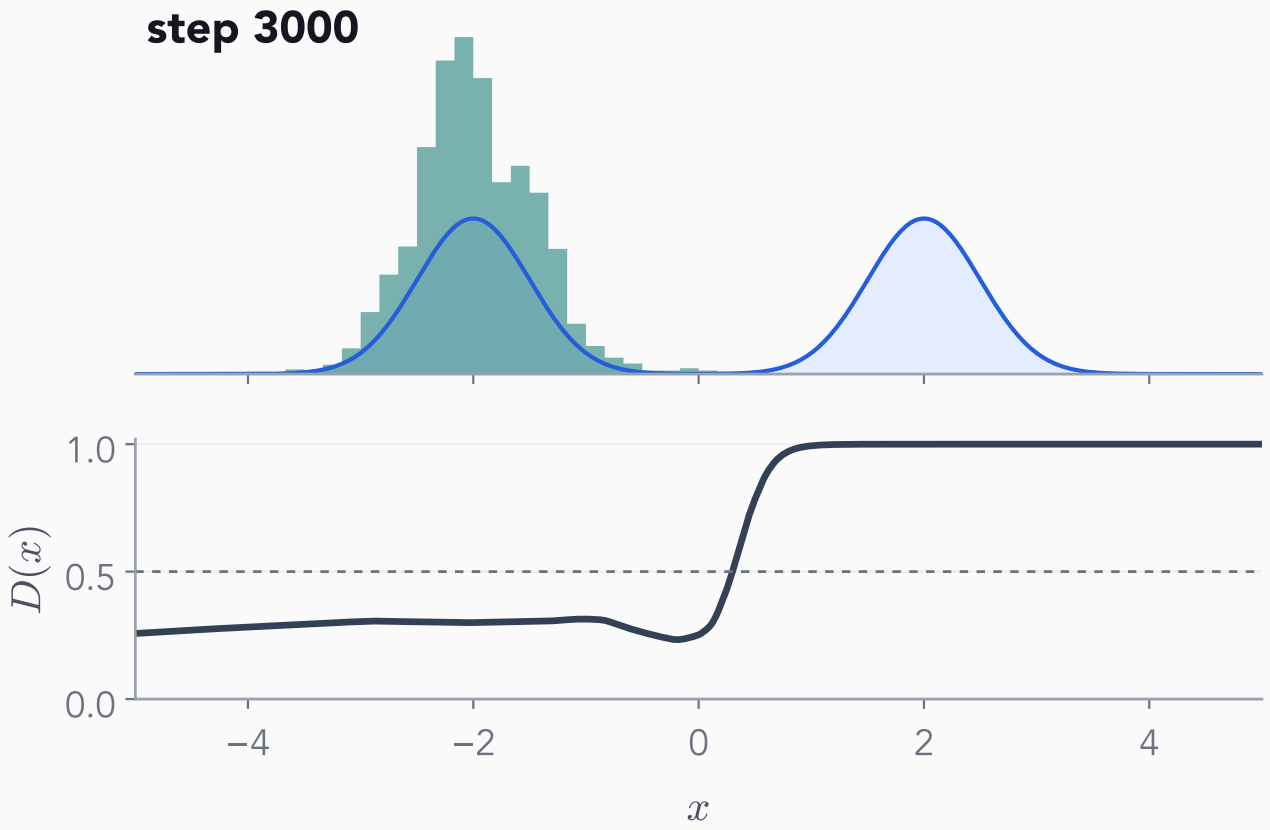


Same data, same training loop, but G has only 4 hidden units per layer.

After 3,000 steps: 95% of samples sit on bump A, 0% on bump B.

THE CLASSIC FAILURE

Mode collapse: perfect samples of only one kind



Same data, same training loop, but G has only 4 hidden units per layer.

After 3,000 steps: 95% of samples sit on bump A, 0% on bump B.

Every sample looks real. Bump B is never generated. Picking your favourite samples would never reveal it.

Like the reverse-KL fit: commit to one mode and ignore the rest.

SO HOW DO YOU KNOW IT WORKED?

The loss curves cannot tell these two runs apart

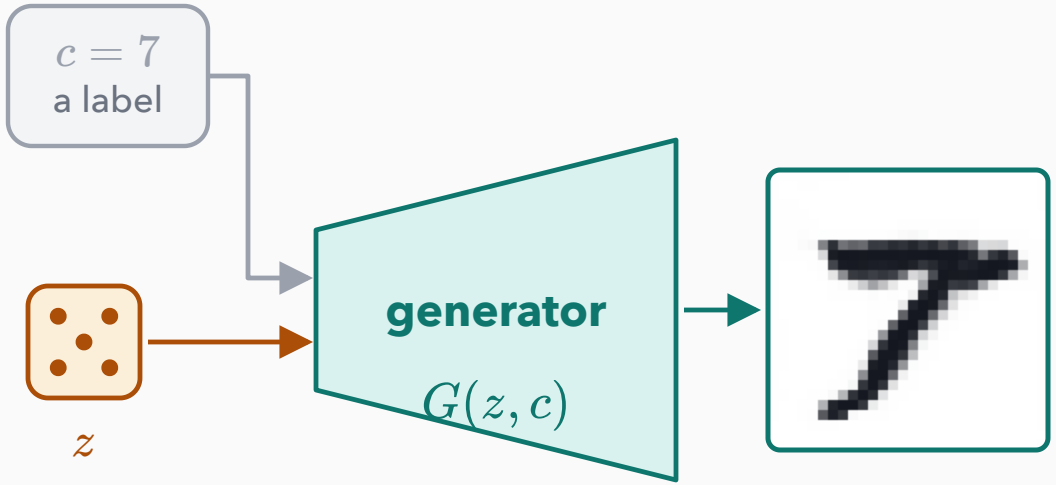




> **Both curves go flat. Only the samples show that one run lost half the data. Always look at samples, from fixed z, across training.**

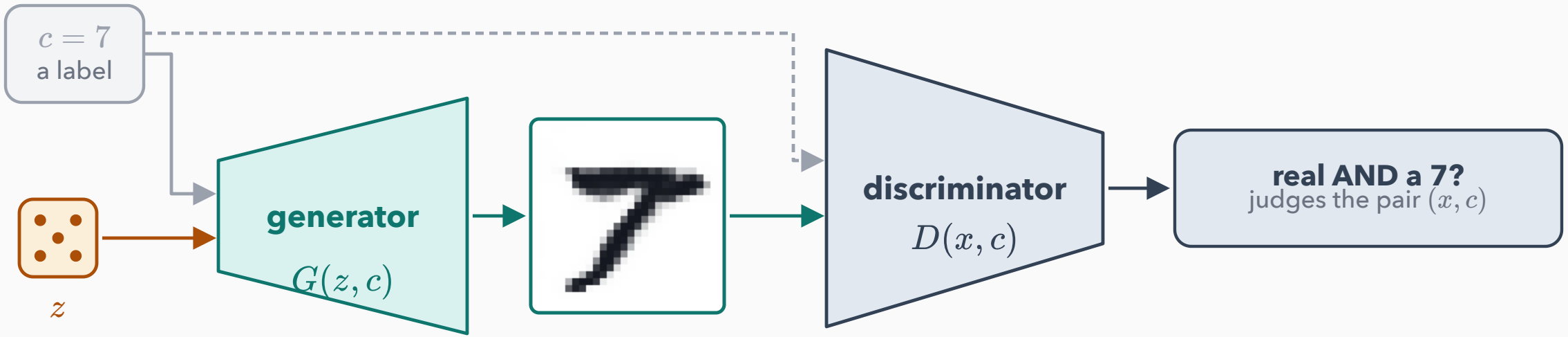
BACK TO THE SHOES

Conditioning: tell both players what you want



BACK TO THE SHOES

Conditioning: tell both players what you want

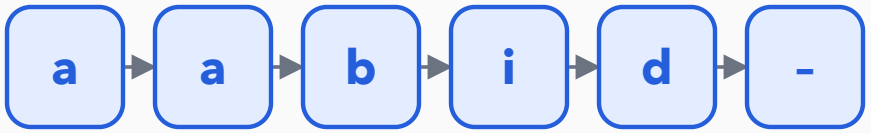


> If D never sees c, G can ignore the request and still fool D. pix2pix conditions on a whole sketch instead of a label.

THREE WAYS TO GENERATE

Same goal, three different training signals

autoregressive
one token at a time

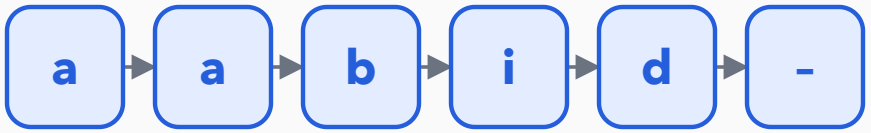


exact likelihood
slow sampling

THREE WAYS TO GENERATE

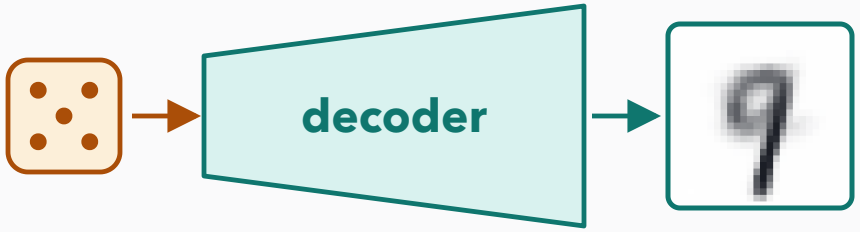
Same goal, three different training signals

autoregressive
one token at a time



exact likelihood
slow sampling

VAE
random z, decode once

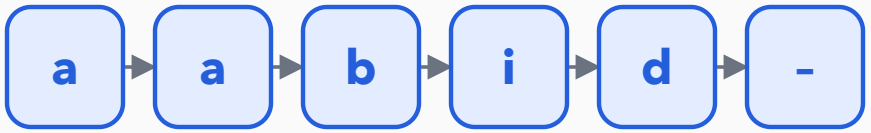


trained on the ELBO
tends to blur

THREE WAYS TO GENERATE

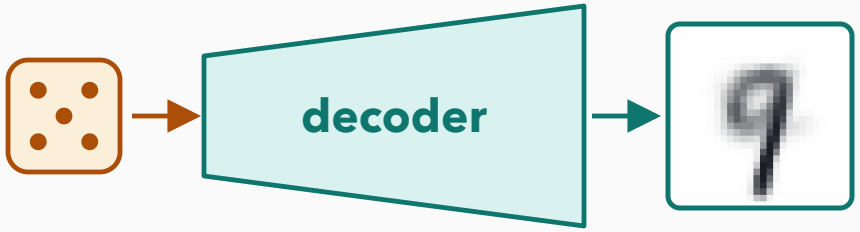
Same goal, three different training signals

autoregressive
one token at a time



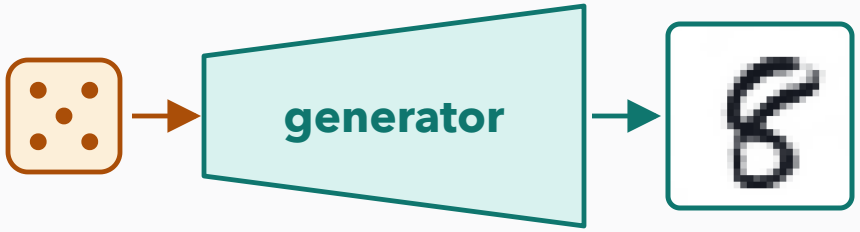
exact likelihood
slow sampling

VAE
random z, decode once



trained on the ELBO
tends to blur

GAN
random z, generate once



trained against a judge
may miss modes

RETURN TO THE OPENING

What you can now explain



a much larger GAN: z → face. Same two-step game, a cleverer G.

RETURN TO THE OPENING

What you can now explain



a much larger GAN: z → face. Same two-step game, a cleverer G.



a conditional GAN: D judges (sketch, photo) pairs, plus an L1 term.

RETURN TO THE OPENING

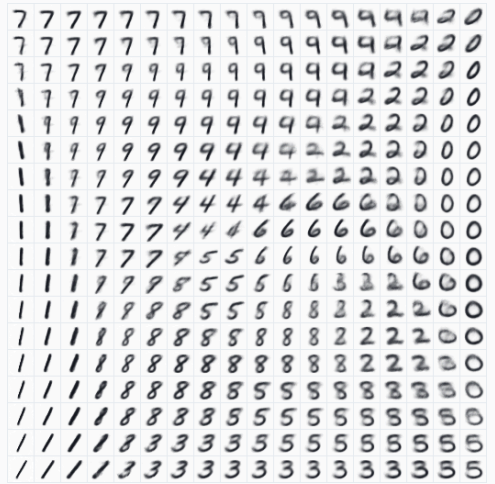
What you can now explain



a much larger GAN: z → face. Same two-step game, a cleverer G.



a conditional GAN: D judges (sketch, photo) pairs, plus an L1 term.



our VAE: every code in $\mathcal{N}(0, I)$ decodes to a digit, thanks to recon + KL.

BEFORE YOU LEAVE

Three exit questions

1. To generate a new digit with a trained VAE, which parts of the network do you use?

2. Two different KL divergences appear in the VAE story. What does each compare, and which one is in the loss?

3. In the G step, D's weights are frozen. How does G still get a gradient?

BEFORE YOU LEAVE

Three exit questions

1. To generate a new digit with a trained VAE, which parts of the network do you use?

   The prior and the decoder only: $z \sim \mathcal{N}(0, I)$, then decode. The encoder is for training.

2. Two different KL divergences appear in the VAE story. What does each compare, and which one is in the loss?

   $\mathrm{KL}(q \,\|\, \text{prior})$ is in the loss (6.97 for our 3). $\mathrm{KL}(q \,\|\, \text{true posterior})$ is the gap between the ELBO and $\log p_\theta(x)$ (0.61).

3. In the G step, D's weights are frozen. How does G still get a gradient?

   Freezing stops D's weights from updating, but the gradient still flows through D's input back into G. Only `detach()` would cut it.

NEXT LECTURE

What if we generate by removing noise, a little at a time?



our 3 → add a little Gaussian noise, again and again → pure noise

> **Adding noise is easy and gives free training pairs. Can a network learn to undo one small step, then walk all the way back from pure noise?**

APPENDIX

Derivations for reference

The optimal discriminator and its link to the Jensen–Shannon divergence. (The Gaussian KL and the ELBO gap are now derived in the main slides.)

APPENDIX C

The optimal discriminator

For fixed G, maximise $\int p_{\text{data}}(x) \log D(x) + p_G(x) \log(1 - D(x))\, dx$ pointwise in $D(x)$:

$$D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_G(x)}$$

Substituting back:

$$V(D^*, G) = -\log 4 + 2\,\mathrm{JS}(p_{\text{data}} \,\|\, p_G)$$

So with a perfect D, G minimises the Jensen–Shannon divergence, which is 0 only when $p_G = p_{\text{data}}$, where $D^* = \frac{1}{2}$ everywhere. Idealised: real D and G are finite networks trained for finitely many steps.

FURTHER READING

Where to go next

- Lilian Weng, *From Autoencoder to Beta-VAE* and *From GAN to WGAN* (lilianweng.github.io)
- Jaan Altosaar, *Tutorial: what is a variational autoencoder?*: the neural-network and probability views side by side
- Kingma & Welling, *Auto-Encoding Variational Bayes*, arXiv 1312.6114
- Goodfellow et al., *Generative Adversarial Nets*, arXiv 1406.2661
- Stanford CS236 notes on VAEs and GANs (deepgenerativemodels.github.io)
- This lecture's code: `scripts/train_models.py` (all models) and `scripts/make_figures.py` (every number on the slides)